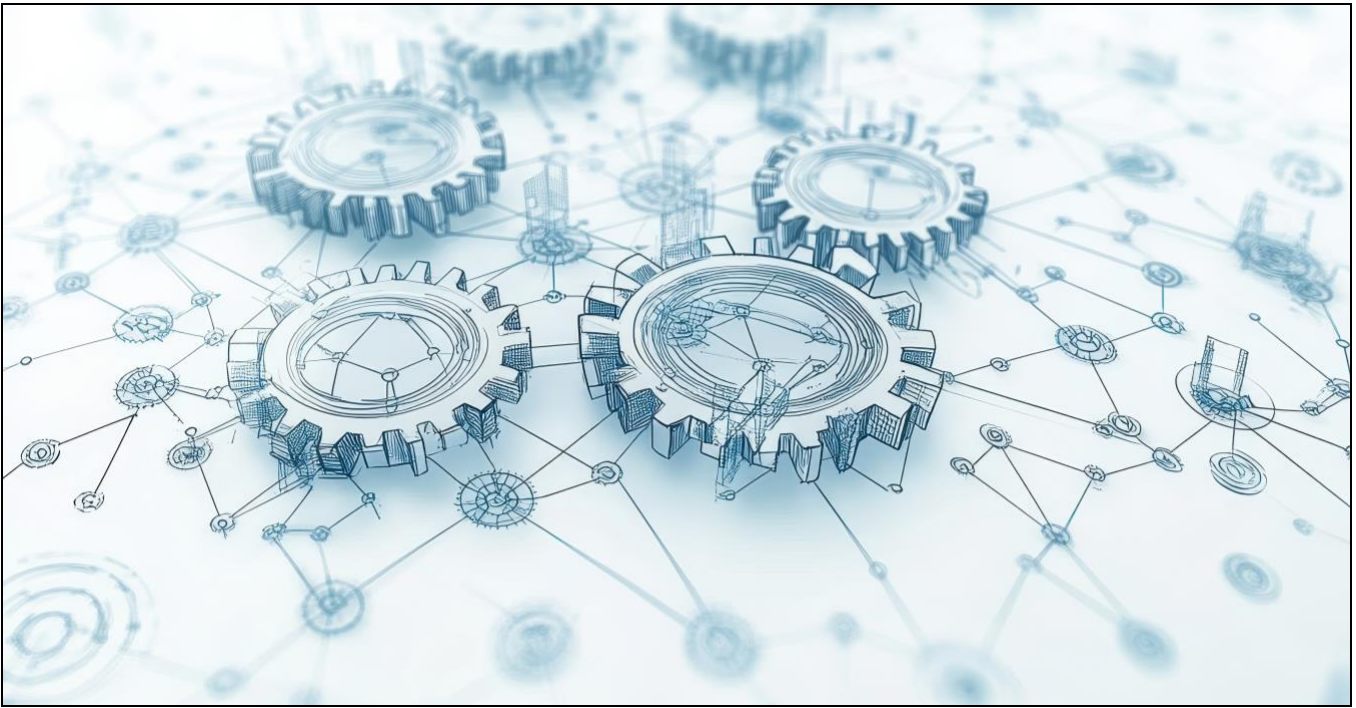


Level 4

Watsonx Orchestrate

Hands-on lab guide: Introduction to Agent Builder



Version 1.4

July 2026

LEGAL NOTICES

This information was developed for products and services offered in the USA. IBM may not offer the products, services, or features discussed in this document in other countries. Consult your local IBM representative for information on the products and services currently available in your area. Any reference to an IBM product, program, or service is not intended to state or imply that only that IBM product, program, or service may be used. Any functionally equivalent product, program, or service that does not infringe any IBM intellectual property right may be used instead. However, it is the user's responsibility to evaluate and verify the operation of any non-IBM product, program, or service. IBM may have patents or pending patent applications covering subject matter described in this document. The furnishing of this document does not grant you any license to these patents. You can send license inquiries, in writing, to:

IBM Director of Licensing IBM Corporation
North Castle Drive, MD-NC119 Armonk, NY 10504-1785
United States of America

The following paragraph does not apply to the United Kingdom or any other country where such provisions are inconsistent with local law: INTERNATIONAL BUSINESS MACHINES CORPORATION PROVIDES THIS PUBLICATION “AS IS” WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. Some states do not allow disclaimer of express or implied warranties in certain transactions; therefore, this statement may not apply to you. This information could include technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in new editions of the publication. IBM may make improvements and/or changes in the product(s) and/or the program(s) described in this publication at any time without notice. Any references in this information to non-IBM websites are provided for convenience only and do not in any manner serve as an endorsement of those websites. The materials at those websites are not part of the materials for this IBM product and use of those websites is at your own risk. IBM may use or distribute any of the information you supply in any way it believes appropriate without incurring any obligation to you. Information concerning non-IBM products was obtained from the suppliers of those products, their published announcements or other publicly available sources. IBM has not tested those products and cannot confirm the accuracy of performance, compatibility or any other claims related to non-IBM products. Questions on the capabilities of non-IBM products should be addressed to the suppliers of those products.

This information contains examples of data and reports used in daily business operations. To illustrate them as completely as possible, the examples include the names of individuals, companies, brands, and products. All names and references for organizations and other business institutions used in this deliverable's scenarios are fictional. Any match with real organizations or institutions is coincidental. All names and associated information for people in this deliverable's scenarios are fictional. Any match with a real person is coincidental.

TRADEMARKS

IBM, the IBM logo, and [ibm.com](https://www.ibm.com) are trademarks or registered trademarks of International Business Machines Corp., registered in many jurisdictions worldwide. Other product and service names might be trademarks of IBM or other companies. A current list of IBM trademarks is available on the web at “Copyright and trademark information” at <https://www.ibm.com/legal/copyright-trademark>.

Adobe, the Adobe logo, PostScript, and the PostScript logo are either registered trademarks or trademarks of Adobe Systems Incorporated in the United States, and/or other countries.

Java and all Java-based trademarks and logos are trademarks or registered trademarks of Oracle and/or its affiliates. INFOBLOX is a trademark of Infoblox Inc. or its affiliated companies, registered in the United States and other countries.

Microsoft®, Azure, Windows, Windows NT, Windows Server, Microsoft® Excel®, Microsoft® Word®, Microsoft® Outlook®, Microsoft® Excel® logo, Microsoft® Word® logo, Microsoft® Outlook® logo, Microsoft® Windows App, Microsoft® Windows App logo and the Windows logo are trademarks of Microsoft® Corporation in the United States, other countries, or both.

Salesforce and Salesforce logo are registered trademarks of SALESFORCE, INC. (CORPORATION; CALIFORNIA, USA).

Slack is a registered trademark of Salesforce, Inc (CORPORATION; DELAWARE, USA).

Red Hat®, JBoss®, OpenShift®, Fedora®, Hibernate®, Ansible®, CloudForms®, RHCA®, RHCE®, RHCSA®, Ceph®, and Gluster® are trademarks or registered trademarks of Red Hat, Inc. or its subsidiaries in the United States and other countries.

Google, Google logo, Google Tasks, Google Cloud Platform, GMAIL, GMAIL logo are registered trademarks of GOOGLE LLC and this material is not endorsed by or affiliated with Google in any way.

Elasticsearch, Elastic search logo, Elastic, Elastic logo are registered trademarks of Elasticsearch BV (LIMITED LIABILITY COMPANY; NETHERLANDS) in United States, other countries or both.

Milvus, Milvus logo are registered trademarks of LF Projects, LLC (LIMITED LIABILITY COMPANY; DELAWARE, USA) in United States, other countries or both.

Meta, Meta logos, Llama, Llama logos are registered trademarks of META PLATFORMS, INC. (CORPORATION; California, USA) in United States, other countries or both.

Mistral, Mistral logo, Mistral AI, Mistral AI logo are registered trademarks of Mistral AI (SOCIÉTÉ PAR ACTIONS SIMPLIFIÉE (SAS); FRANCE) in United States, other countries or both.

PyCharm, PyCharm logo are registered trademarks of JetBrains s.r.o. (CORPORATION; CZECH REPUBLIC) in United States, other countries or both.

Bruno is a trademark held by Anoop M D. Bruno logo is sourced from OpenMoji. License: CC BY-SA 4.0

Apple, Apple logo, Apple store, App store, App store logo are registered trademarks of Apple Inc. (CORPORATION; CALIFORNIA, USA) in United States, other countries or both.

VMWARE, VMWARE logos are registered trademarks of VMware LLC (LIMITED LIABILITY COMPANY; Delaware, USA) in United States, other countries or both.

RANCHER, RANCHER logo, Rancher Desktop, Rancher Desktop logo are registered trademarks of SUSE LLC (LIMITED LIABILITY COMPANY; MASSACHUSETTS, USA) in United States, other countries or both.

Langfuse, Langfuse logo are registered trademarks of Langfuse GmbH (GESELLSCHAFT MIT BESCHRÄNKTER HAFTUNG (GMBH); Germany, Germany) in United States, other countries or both.

OpenMeteo is a registered trademark owned by Patrick Zippenfenig (Individual, Germany) in United States, other countries or both.

GROQ is a trademark or registered trademark of Groq, Inc. in the U.S. and/or other countries.

© Copyright International Business Machines Corporation 2025. This document may not be reproduced in whole or in part without the prior written permission of IBM.

Table of Contents

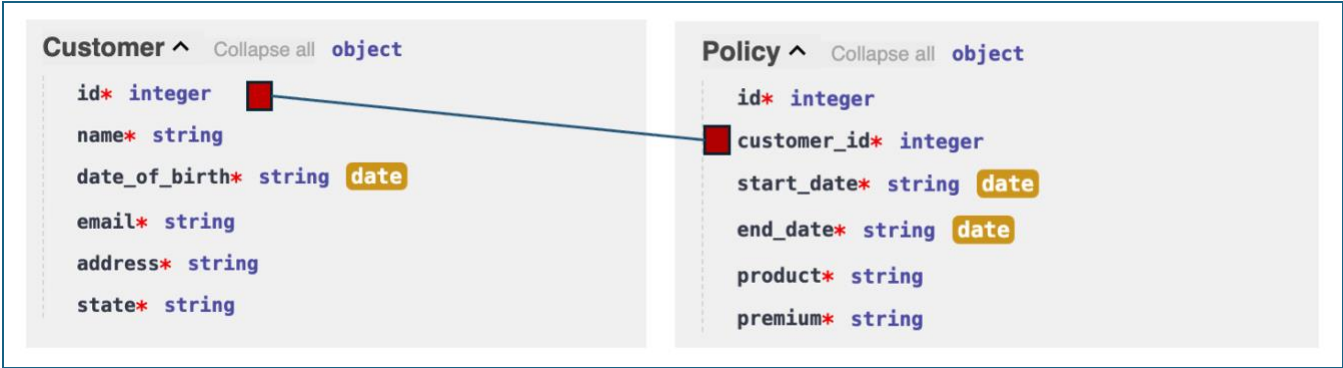
- Introduction 1
 - Lab overview.....1
 - What you will learn2
 - Environment2
 - How to get support.....2
 - Prerequisites2
- Lab files 3
- Insurance APIs..... 5
- Agent creation..... 11
- Tool import..... 16
- Basic tool usage 22
- Filtering..... 24
- Aggregation 31
- Context..... 39
- Agent deployment..... 44
- Agent analytics..... 47
- Summary 50

Introduction

Lab overview

In this lab you're going to be using agent builder to create a simple AI agent that can help brokers in a hypothetical insurance brokerage.

We'll start by looking at the data model and underlying APIs. For simplicity we are only dealing two entities, Customer and Policy, and they are linked using the customer ID.



The platform is API based and exposes the following endpoints that will be imported into Orchestrate as tools for the agent.

GET	/customers	Customers	▼
GET	/customers/{customer_id}	Customers by ID	▼
GET	/policies/{policy_id}	Policies by ID	▼
GET	/customers/{customer_id}/policies	Customer policies	▼
GET	/customers/state/{state}	Customers by State	▼
GET	/customers/{customer_id}/total-premium	Total premium	▼
GET	/policies/{policy_id}/days-to-end	Calculate days left on policy	▼

Once imported into Orchestrate we'll see how these tools are used by the agent to respond to our requests. Next, we'll look at filtering, aggregation, and the importance of memory when using the agent, we'll finish by reviewing agent analytics.

What you will learn

In this lab, you will explore how to:

- Build an agent equipped with tools that interact with external systems
- Refine and format the data your agent displays
- Filtering and aggregation with agents
- Understand the role of agent memory for problem solving
- Use agent instructions to improve accuracy and usability
- Deploy your agent
- Review agent analytics

By the end of this lab, you will be able to confidently apply these skills to create simple user-friendly agents using Agent Builder and have the foundational knowledge needed to create more sophisticated agents.

Environment

This lab has been tested with Agent Builder using watsonx Orchestrate SaaS running on IBM Cloud.

How to get support

If you encounter an issue with this lab,

- IBM employees can open an IBM Support ticket at <http://ibm.biz/support>.
- Business Partners can request help at the [Partner Plus Support](#) website.



When using a large language model (LLM) for reasoning, planning and problem solving, results can vary. The same instructions can produce very different results, even with the same input, so expect some differences when using your agent. Some adaptation may be required.

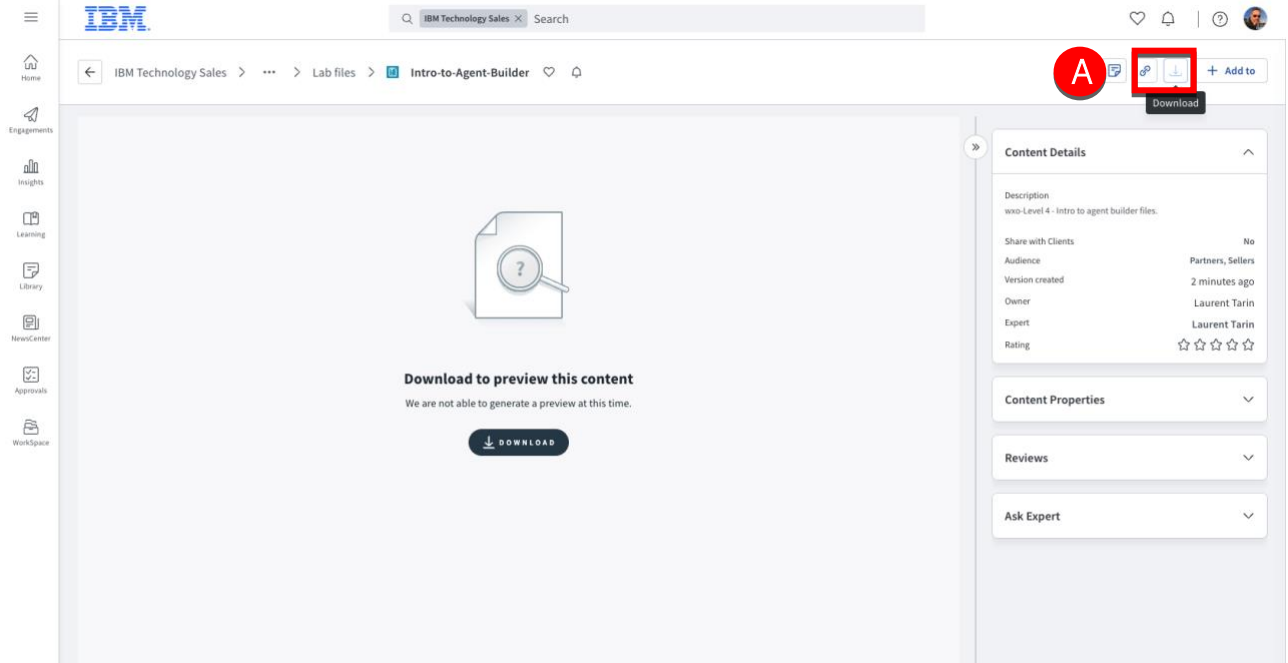
Prerequisites

- A basic understanding of agentic AI.
- You have completed the steps in wxO L4 preparation guide included with this course.

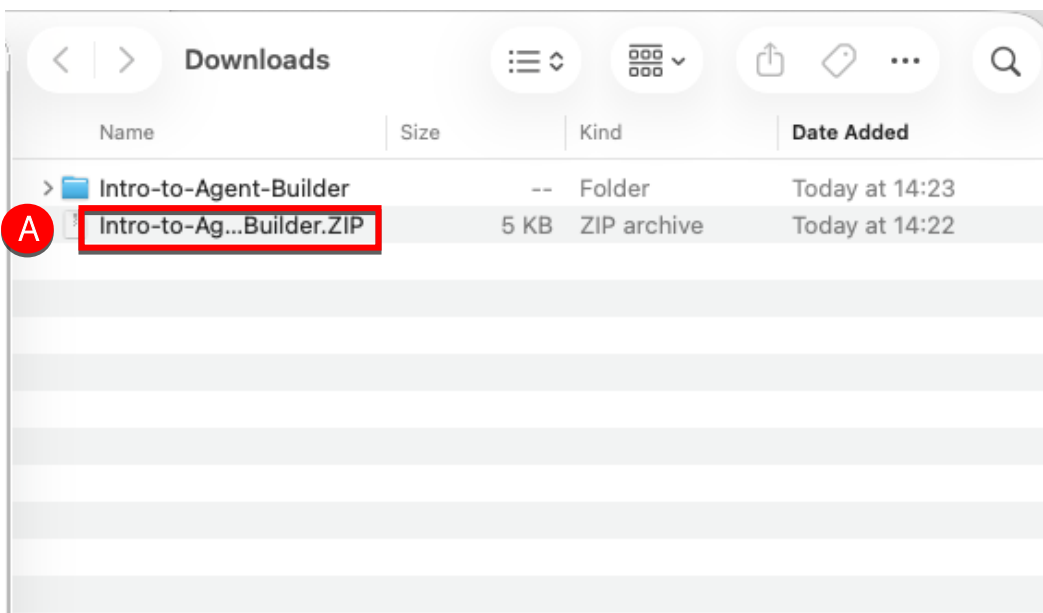
Lab files

The files required to complete this lab are hosted on Seismic, you will need to download these files.

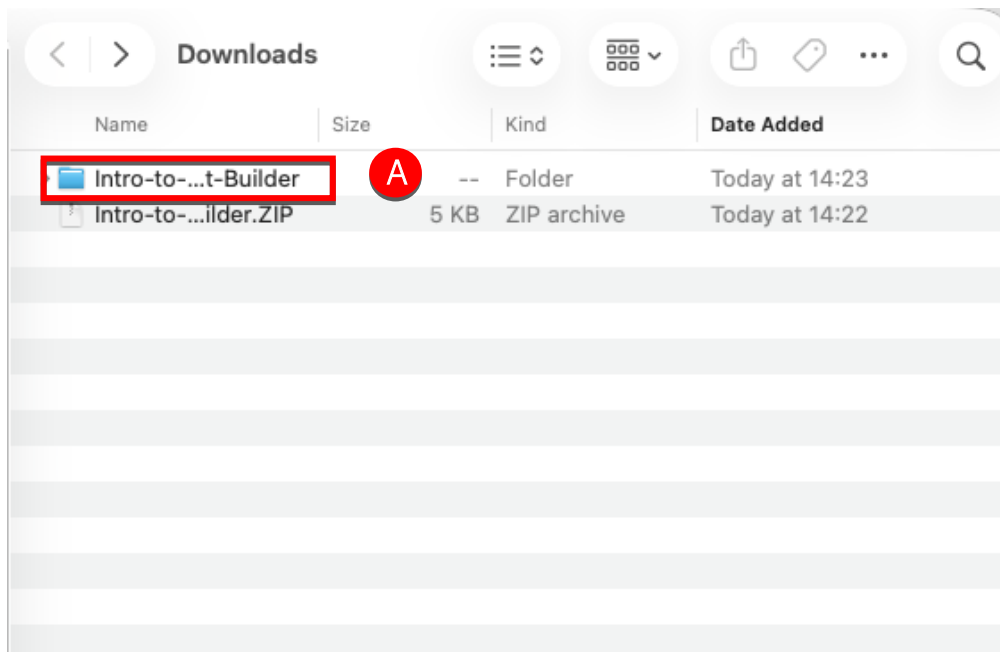
1. To obtain the files needed for this lab open this URL in your browser:
2. Get the archive on <https://ibm.seismic.com/Link/Content/DC4Wd9hjHQJcVG4RdJ7Q2g29FmJd>
3. Click the download icon (A).



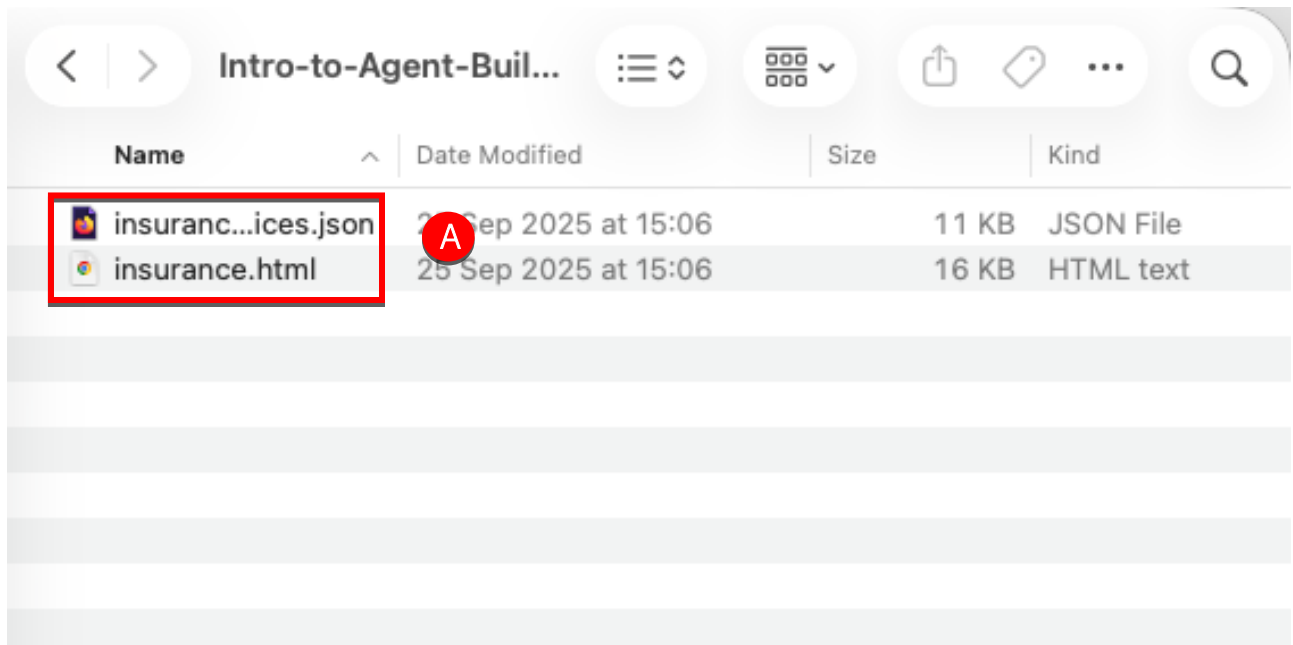
4. Extract the intro-to-Agent-Builder.zip archive (A) using the appropriate file extractor.



5. Open **Intro-to-Agent-Builder** (A).



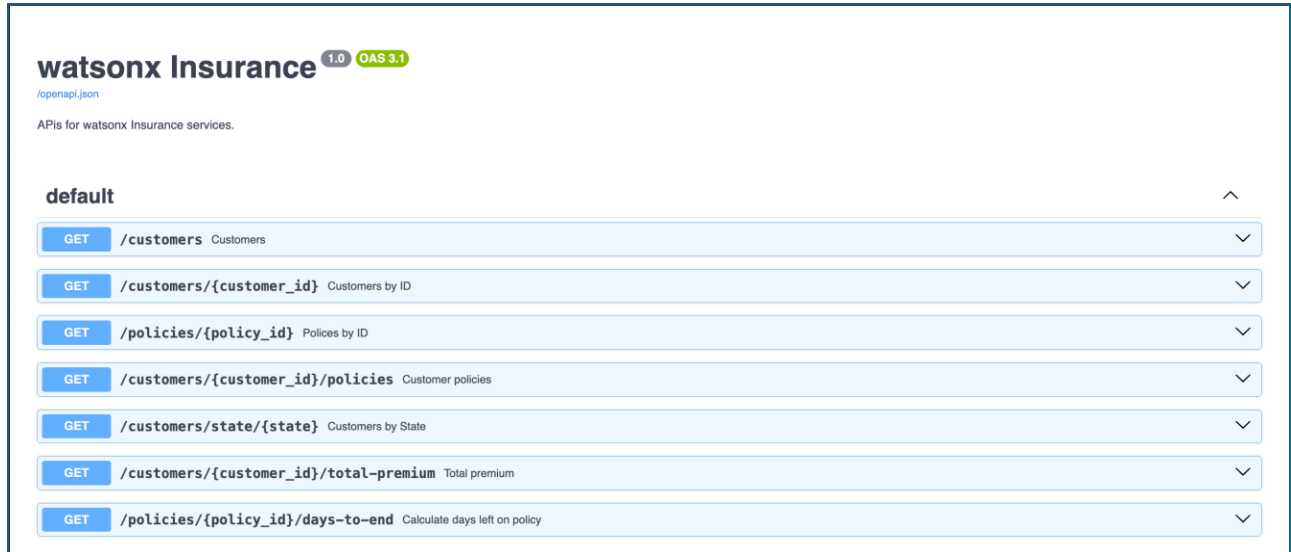
6. Check that the 2 required files are on your local computer (A).



Insurance APIs

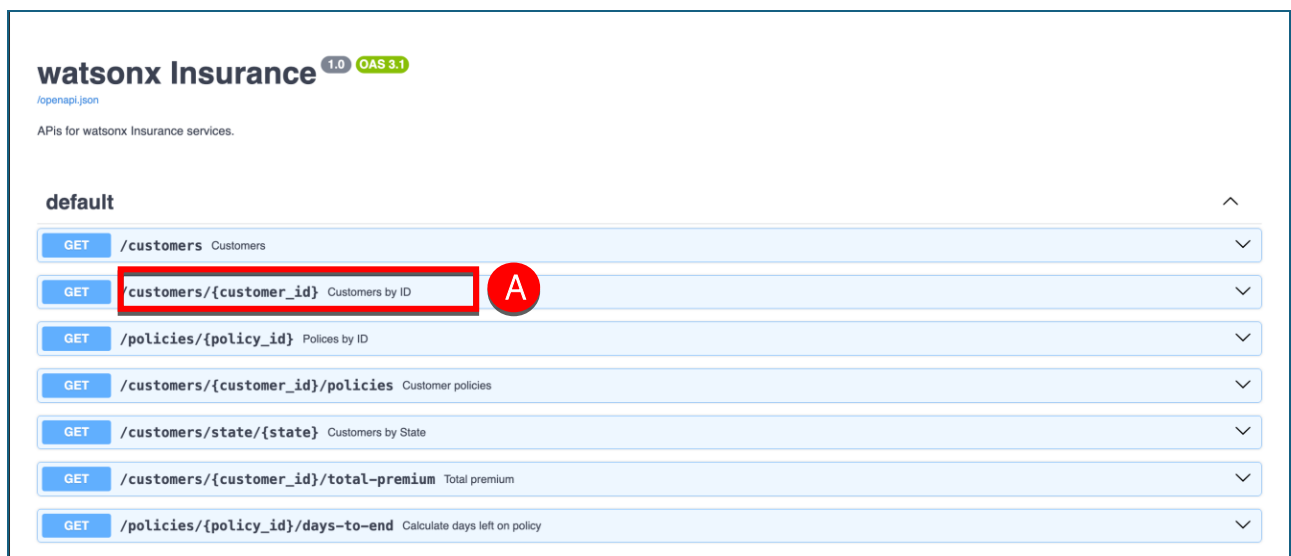
In this section you will familiarize yourself with the API's that will be used to create the tools in Agent Builder.

1. In your web browser, go to <https://l4-insurance.1d13bpwyy9q7.us-east.codeengine.appdomain.cloud/docs>.

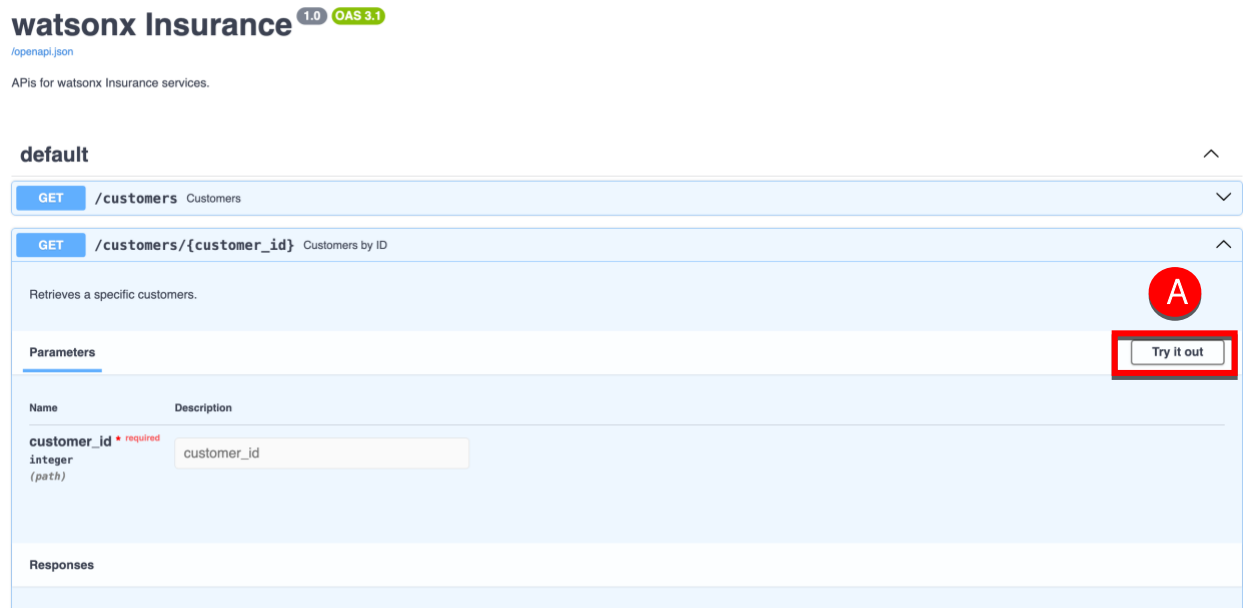


This is the application that provides the API endpoints that we'll use to create tools in Orchestrate. There are a small number of operations available allowing us to retrieve information about customers and policies. Let's test the operations we'll be using with our agent.

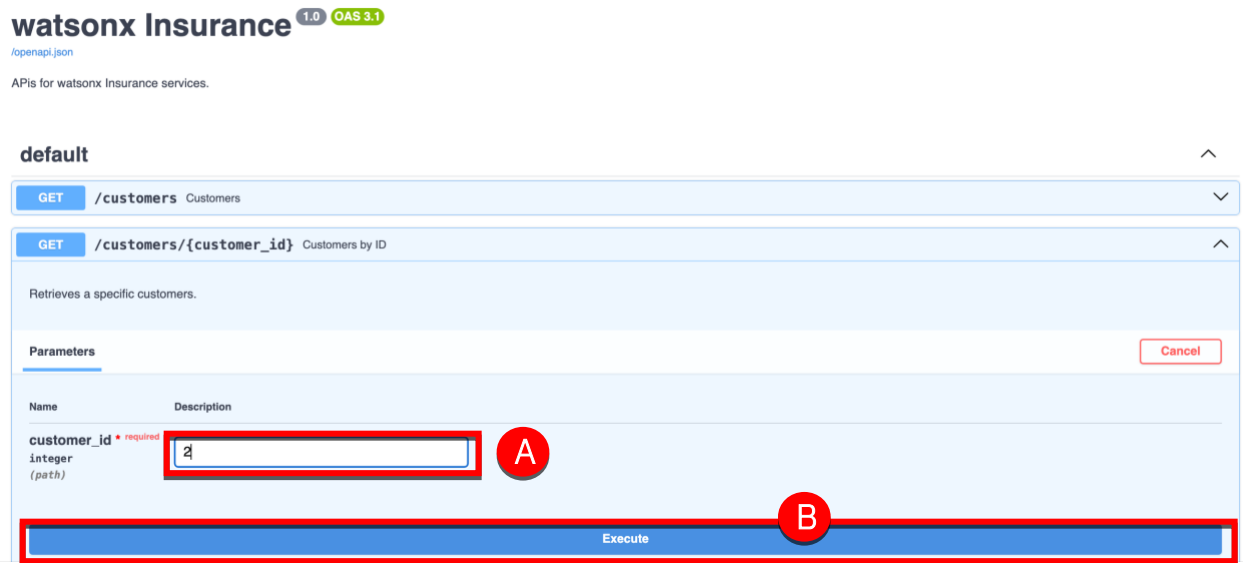
2. Click **Customers by ID (A)**.



- Click **Try it out** (A).



- Enter 2 into the **customer_id** field (A), then click **Execute** (B).



5. Scroll down (A) until the **Response body** (B) is displayed.

The screenshot shows an API client interface with the following sections:

- Responses** (header)
- Curl**: `curl -X 'GET' \ 'https://l4-insurance.1d13bpwy9q7.us-east.codeengine.appdomain.cloud/customers/2' \ -H 'accept: application/json'`
- Request URL**: `https://l4-insurance.1d13bpwy9q7.us-east.codeengine.appdomain.cloud/customers/2`
- Server response**:
 - Code**: 200
 - Details**:
 - Response body** (highlighted with a red box labeled 'B'):

```
{
  "id": 2,
  "name": "Sarah Johnson",
  "date_of_birth": "1990-03-22",
  "email": "sarah.johnson@email.com",
  "address": "456 Oak Ave",
  "state": "Texas"
}
```
 - Response headers**:

```
content-length: 134
content-type: application/json
date: Wed, 24 Sep 2025 09:10:13 GMT
server: uvicorn
strict-transport-security: max-age=63072000; preload
```
- Responses** (table):

Code	Description	Links
200	Successful Response	No links
- Media type**: `application/json`

This shows the customer data held in the system for customer 2. To use this endpoint, Orchestrate will need to analyze our request and extract the customer ID before passing it as a parameter to the API.

6. Return to the top of the web page and click **Customers by ID** (A) to collapse this section.

The screenshot shows the Watsonx Insurance API Explorer interface with the following sections:

- watsonx Insurance** (1.0 OAS 3.1)
- /openapi.json**
- APIs for watsonx Insurance services.**
- default** (header)
- GET /customers** Customers
- GET /customers/{customer_id}** Customers by ID (highlighted with a red box labeled 'A')
 - Retrieves a specific customers.
 - Parameters**:

Name	Description
customer_id * required	
integer	
(path)	
 - Execute** button

- Click **Customer Policies** (A) to expand it, then click **Try it out** (B).

watsonx Insurance ^{1.0} **OAS 3.1**
/openapi.json
APIs for watsonx Insurance services.

default ^

- GET **/customers** Customers ^
- GET **/customers/{customer_id}** Customers by ID ^
- GET **/policies/{policy_id}** Policies by ID ^
- GET **/customers/{customer_id}/policies** Customer policies ^

Retrieves policy information for a specific customer.

Parameters

Try it out

A **B**

- Enter 2 in the **customer_id** field (A), then click **Execute** (B)

default ^

- GET **/customers** Customers ^
- GET **/customers/{customer_id}** Customers by ID ^
- GET **/policies/{policy_id}** Policies by ID ^
- GET **/customers/{customer_id}/policies** Customer policies ^

Retrieves policy information for a specific customer.

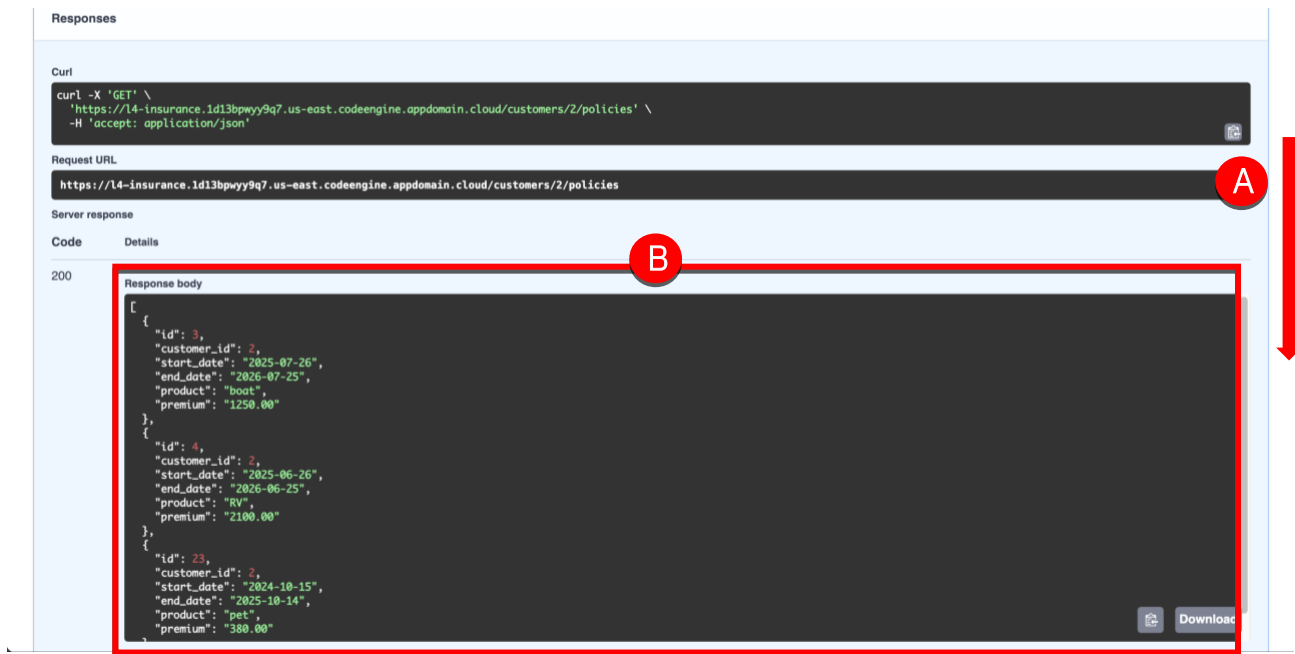
Parameters Cancel

Name	Description
customer_id * required	
integer	
(path)	

Execute

A **B**

9. Scroll down (A) until the **Response body** (B) is displayed.



The screenshot shows a REST client interface with the following sections:

- Responses**: A tab is selected.
- Curl**: A text box containing the command: `curl -X 'GET' \ 'https://l4-insurance.1d13bpwy9q7.us-east.codeengine.appdomain.cloud/customers/2/policies' \ -H 'accept: application/json'`
- Request URL**: A text box containing: `https://l4-insurance.1d13bpwy9q7.us-east.codeengine.appdomain.cloud/customers/2/policies`. A red circle labeled 'A' is next to it.
- Server response**: A section with a **Code** tab selected, showing a status code of **200**.
- Response body**: A large text area containing a JSON array of three policy objects. A red circle labeled 'B' is at the top center of this area. A red arrow points downwards from the 'Request URL' section towards the 'Response body'.

The JSON response body is:

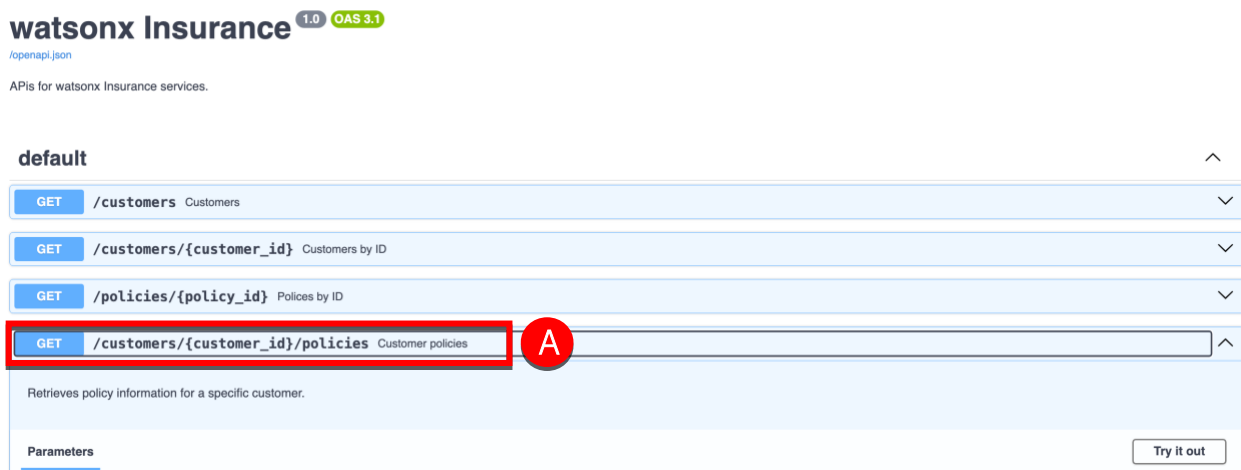
```
[
  {
    "id": 3,
    "customer_id": 2,
    "start_date": "2025-07-26",
    "end_date": "2026-07-25",
    "product": "boat",
    "premium": "1250.00"
  },
  {
    "id": 4,
    "customer_id": 2,
    "start_date": "2025-06-26",
    "end_date": "2026-06-25",
    "product": "RV",
    "premium": "2100.00"
  },
  {
    "id": 23,
    "customer_id": 2,
    "start_date": "2024-10-15",
    "end_date": "2025-10-14",
    "product": "pet",
    "premium": "380.00"
  }
]
```

This shows the policy information held in the system for customer 2.



Note that the policy start and end dates are regenerated based on today's date, so the results you see will be different.

10. Return to the top of the web page and click **Customer policies** (A) to collapse this section.



The screenshot shows the Watsonx Insurance API Explorer interface:

- watsonx Insurance**: Version 1.0, OAS 3.1.
- /openapi.json**: Link to the OpenAPI specification.
- APIs for watsonx Insurance services.**
- default**: A section header with an expand/collapse icon.
- Endpoints**: A list of endpoints with their methods and descriptions:
 - GET /customers**: Customers
 - GET /customers/{customer_id}**: Customers by ID
 - GET /policies/{policy_id}**: Policies by ID
 - GET /customers/{customer_id}/policies**: Customer policies (highlighted with a red box and a red circle labeled 'A')
- Description**: "Retrieves policy information for a specific customer."
- Parameters**: A section header.
- Try it out**: A button.

The remaining operations (A), (B), and (C), filter or perform calculations:

- (A) **Customers by State**: searches for customers in a specific state.
- (B) **Total premium**: Calculates the total premiums paid by a specific customer.
- (C) **Calculate days left on policy**: Calculates the number of days until a specific policy ends.



1. Try Customers by State (A) using Texas as a parameter.

This operation retrieves all customers in a specific state; the operation is case sensitive.

2. Try Total premium (B) using 2 as a parameter.

This operation is used to calculate the total premium of all policies held by a customer.

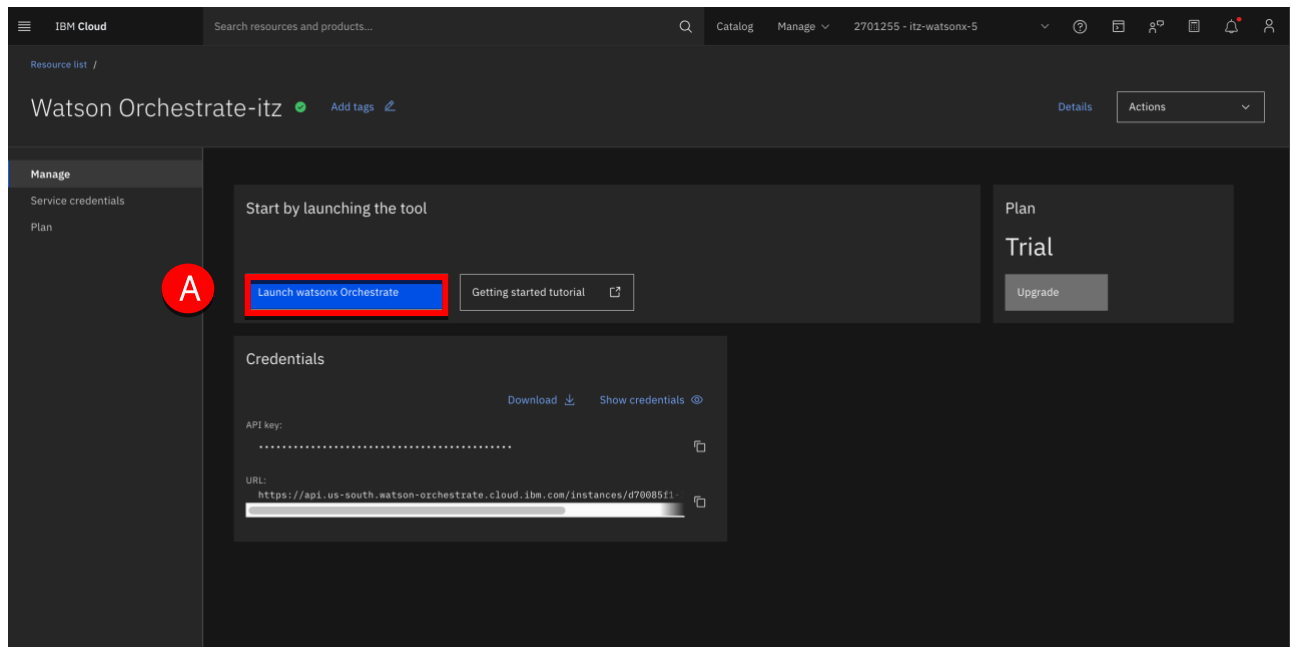
3. Try Calculate days left on policy (C) using 11 as a parameter.

This operation calculates the number of days before a policy expires.

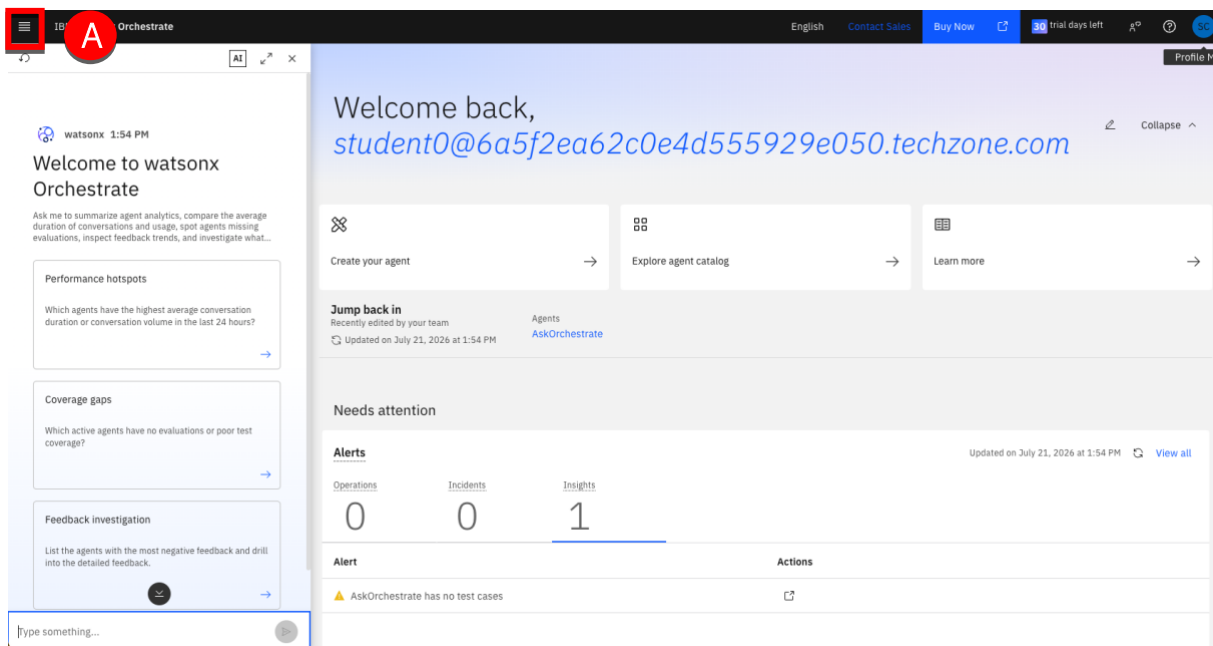
Agent creation

In this section we will create the agent provide a description of its responsibilities.

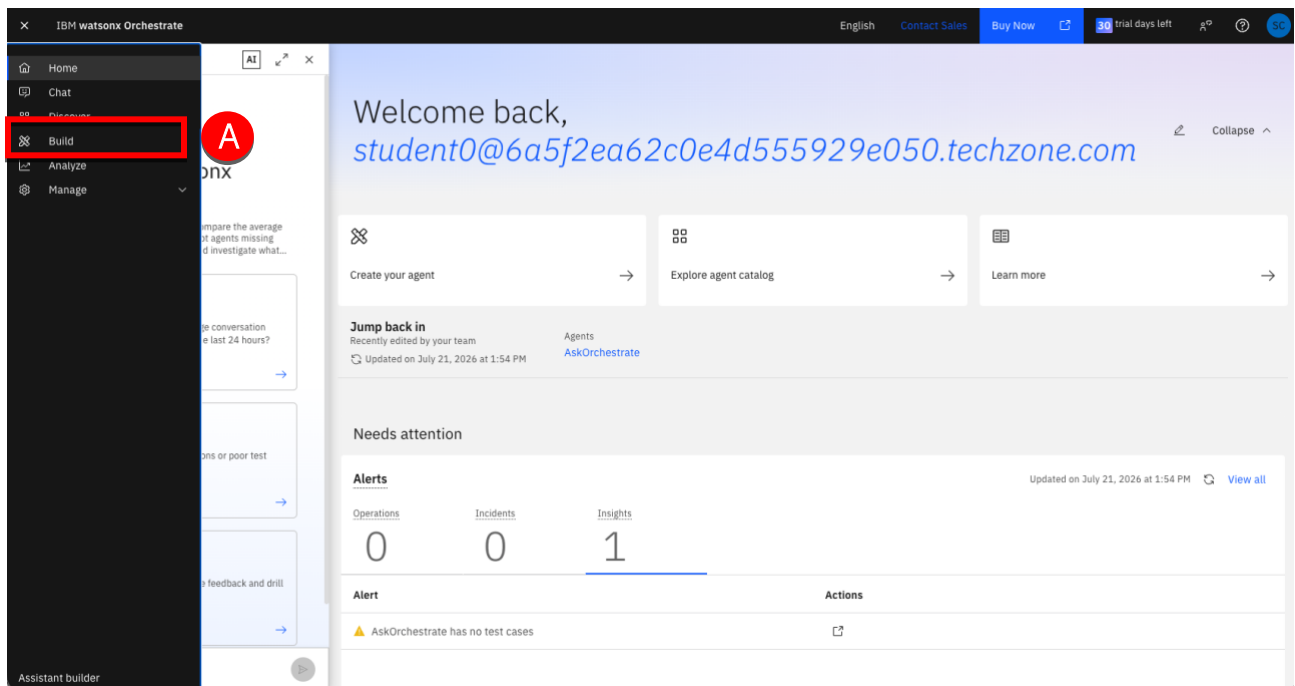
1. From your IBM Cloud service, click **Launch watsonx Orchestrate** (A).



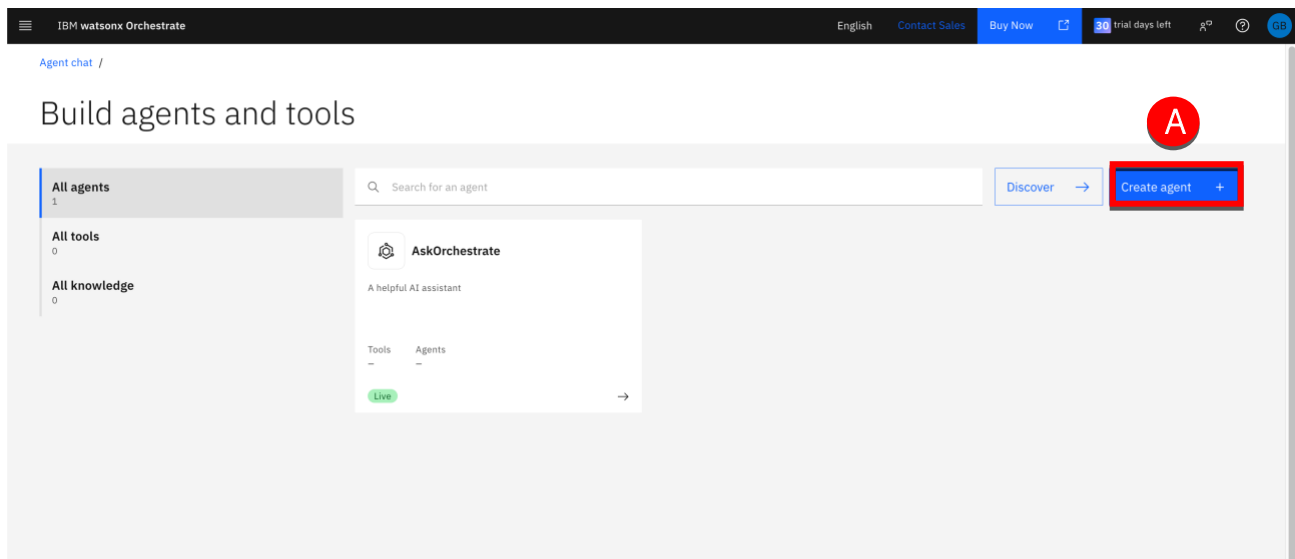
2. Click the main menu icon in the top-left corner (A).



3. Click **Build** (A).



4. Click **Create agent** (A).



5. Enter “Insurance” into the **Name** field (A), then enter “This agent retrieves information about customers and their policies.” into the **Description** field (B) and click **Create** (C).

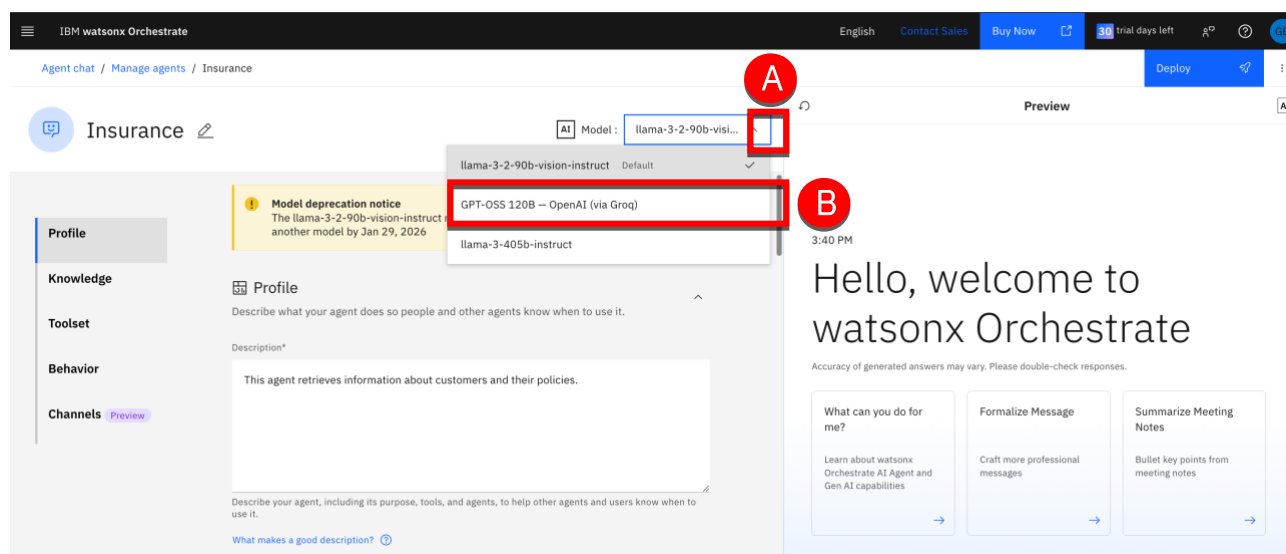
The Agent Builder opens; the screen is divided into three key areas:

- **Navigation Panel (A)**: move between different sections of Agent Builder.
- **Configuration (B)**: set up and customize the functionality of your agent.
- **Preview (C)**: preview and test your agent.

Below is a description of each section:

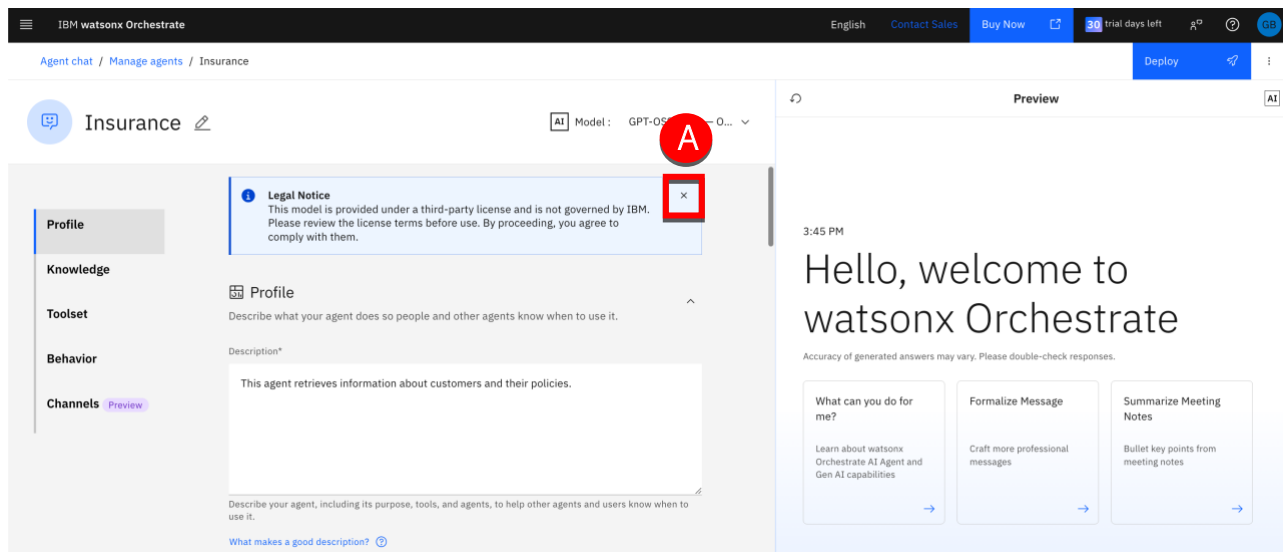
- **Profile:** Define your agent’s purpose, usage scenarios, and interaction style. Describe what the agent does, when it should be used (especially in multi-agent configurations), and choose its style—Default or ReAct—to guide how it interprets requests, plans and uses tools.
- **Knowledge:** This is where you equip your agent with knowledge by uploading files or connecting to conversational search platforms such as Milvus, Elasticsearch, or custom-built sources. This ensures the agent can generate accurate, contextual responses by drawing on relevant content.
- **Toolset:** Provide your agent with tools to perform tasks. Tools can be added from the Catalog, imported from OpenAPI specification files and MCP servers, or built with custom flows. Tools extend the agent’s capabilities, enabling it to automate actions such as retrieving data or sending emails.
- **Behavior:** Here, you define how the agent interacts with users, formats data and how it handles requests. Add rules and instructions to shape its tone, response style, and overall behavior during interactions.
- **Channels:** This is where you connect your agent to communication platforms like Slack or embed the agent in a website.

6. Click the down-arrow (A) and select **GPT-OSS 120B – OpenAI (via Groq)** from the list (B).



Note: If the model selection is disabled on your instance, you can turn it on from the Settings menu accessible from your profile on the top right of the screen and then navigate to the Models tab.

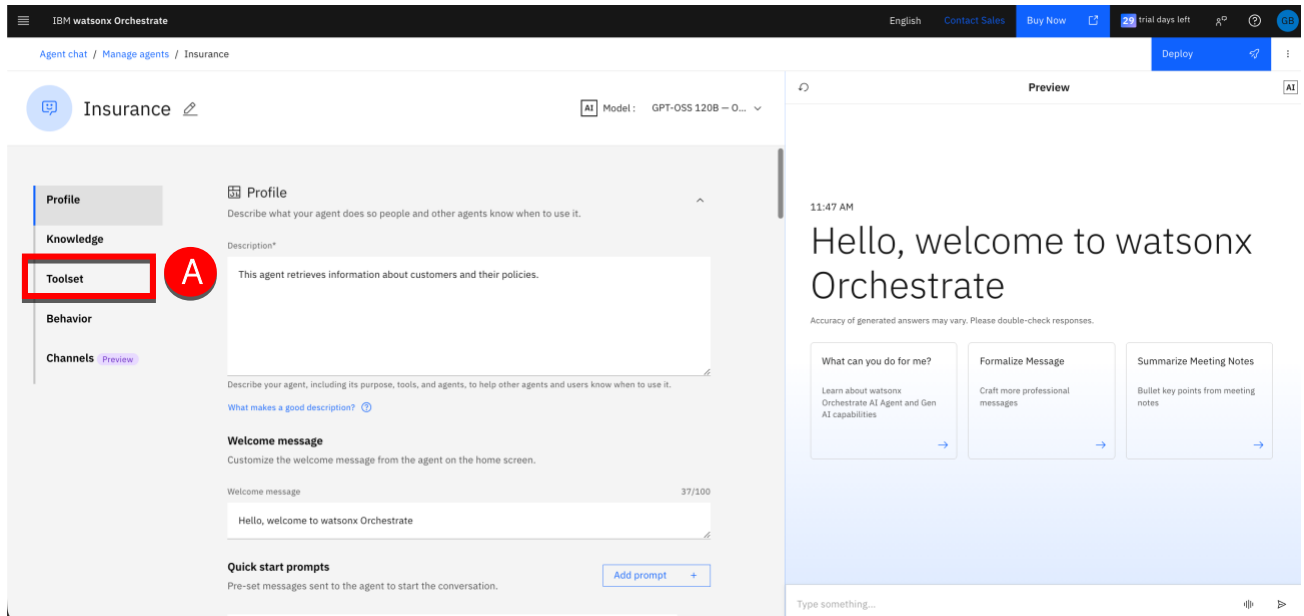
7. Click the cross (A) to close the Legal Notice.



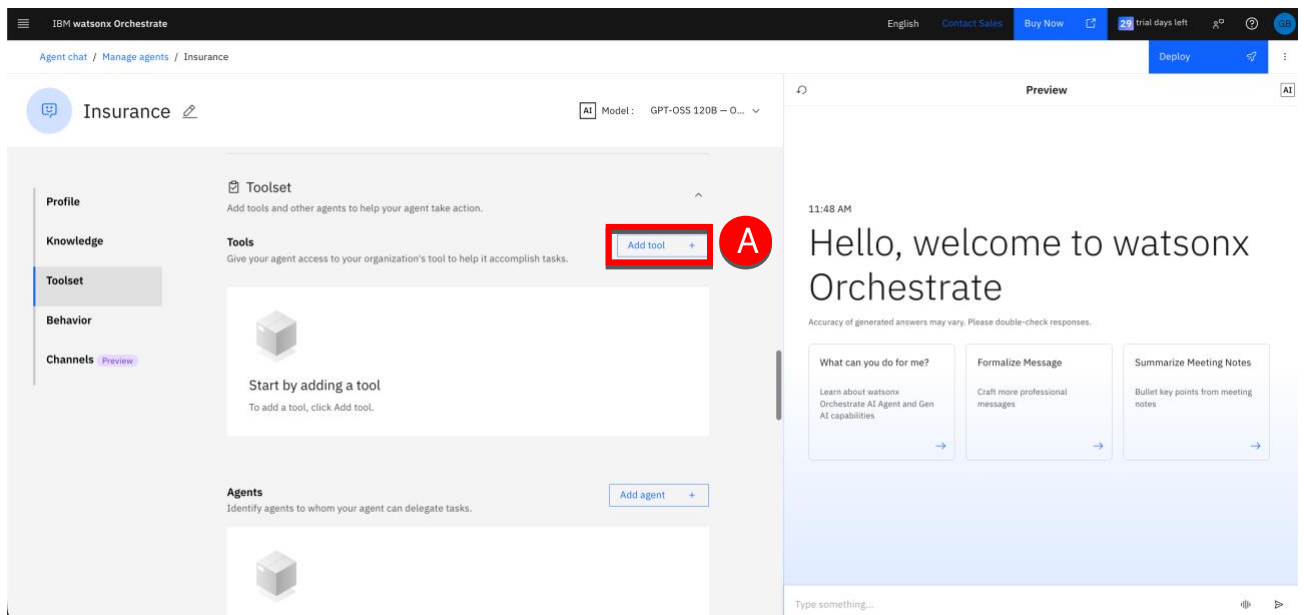
Tool import

In this section we will create the tools used by the agent.

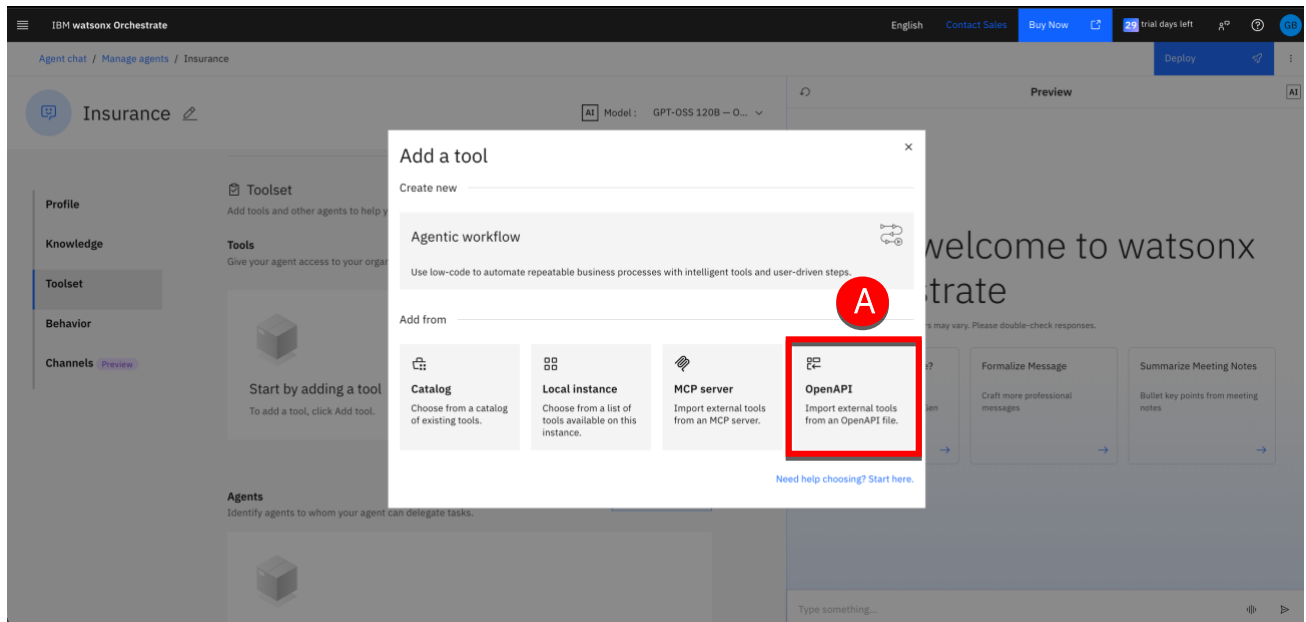
1. Click **Toolset** (A).



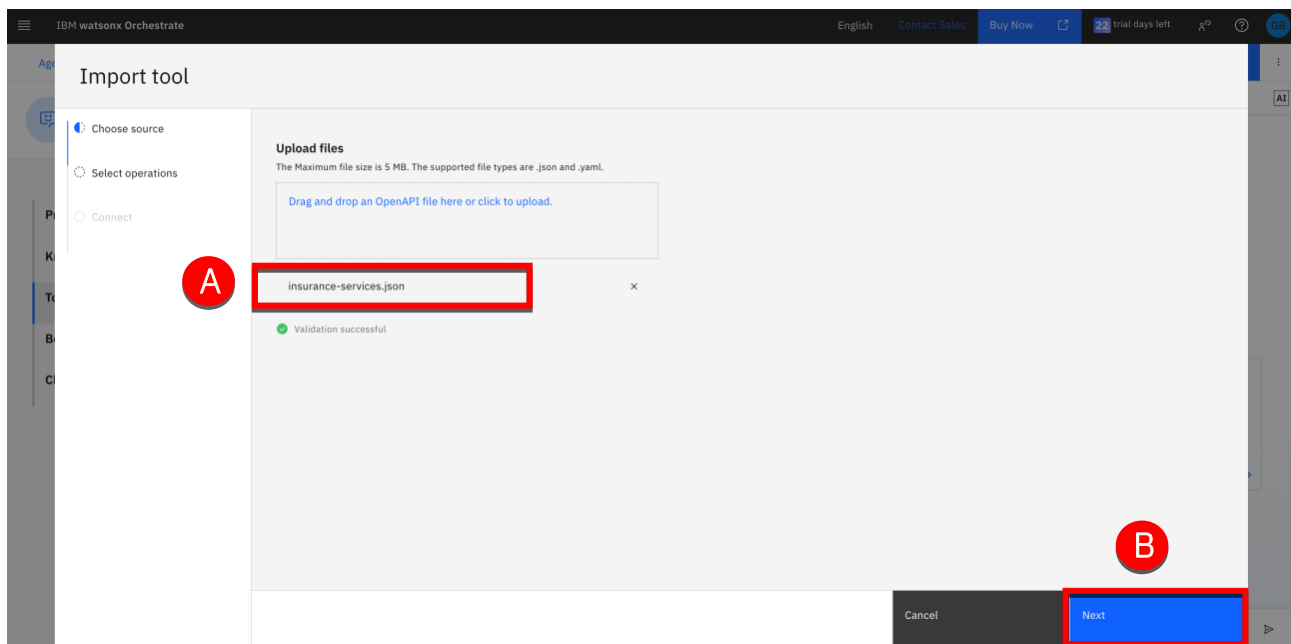
2. Click **Add tool** (A).



3. Click **OpenAPI** (A).

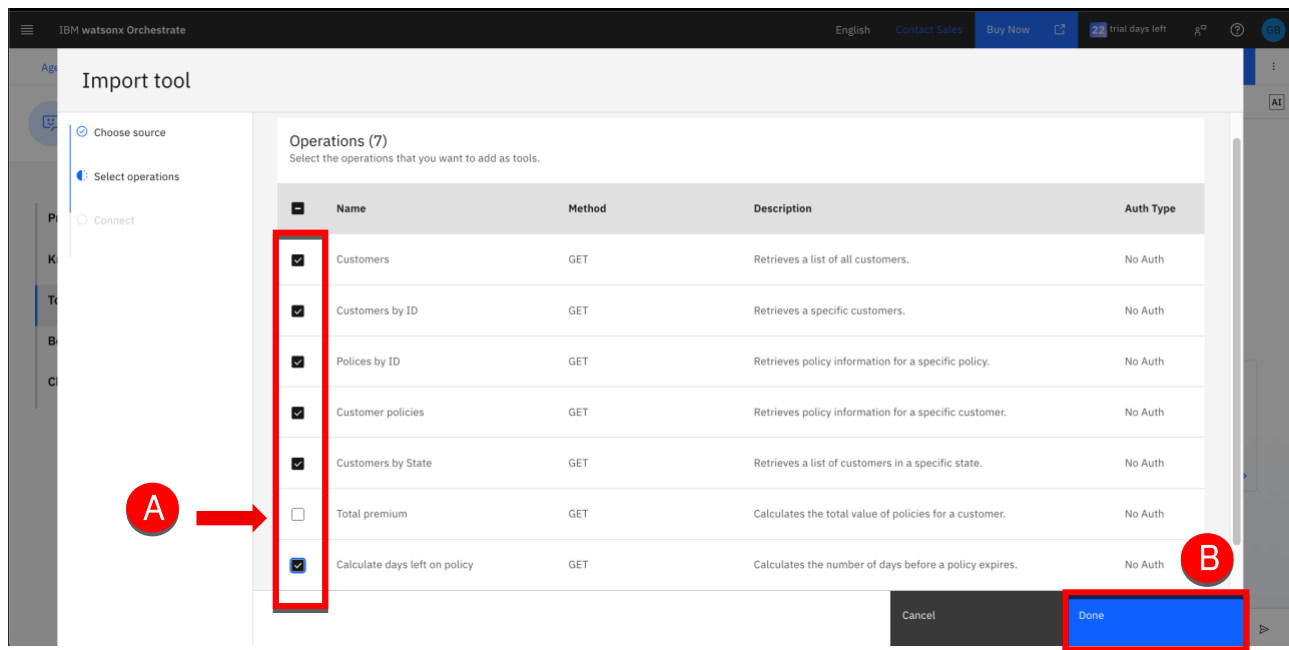


4. Upload or drag and drop the **insurance-service.json** file (A), then click **Next** (B).



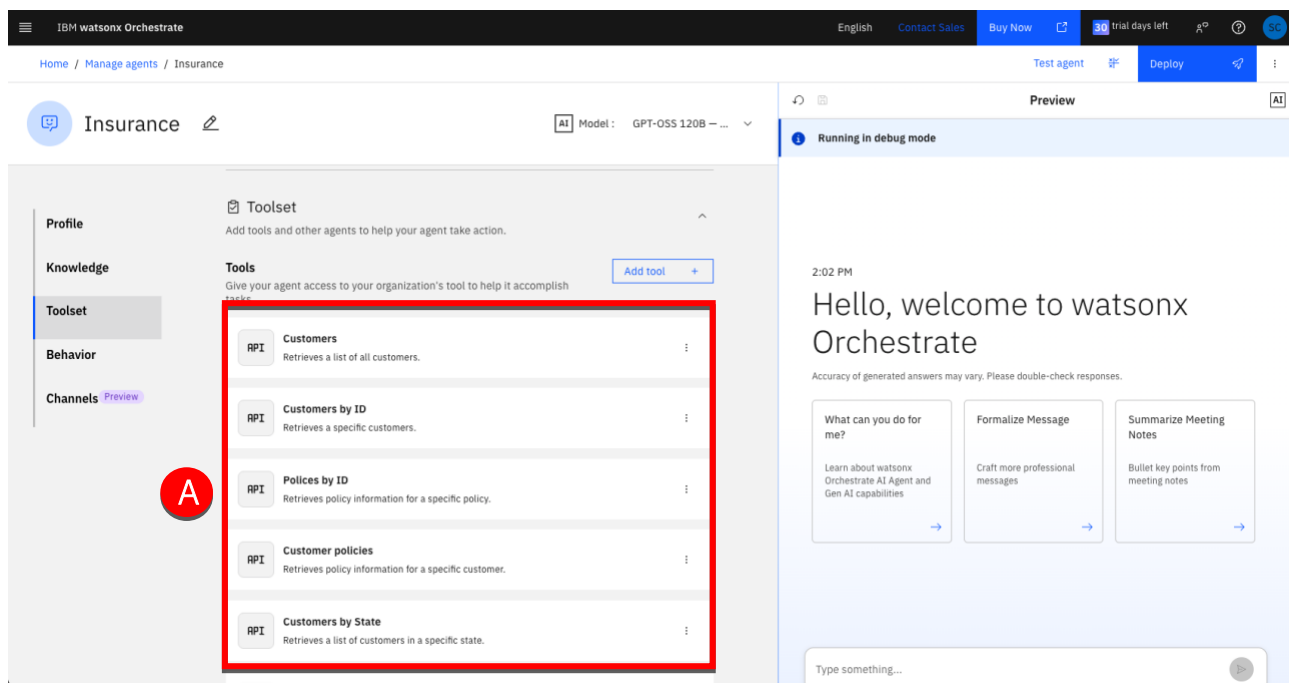
This file contains the OpenAPI specification for the API and was downloaded from the API user interface. Extensive OAS resources are available online, this link from the OpenAPI initiative provides further details (<https://www.openapis.org/what-is-openapi>).

5. Select all operations except for **Total premium** (A), then click **Done** (B)

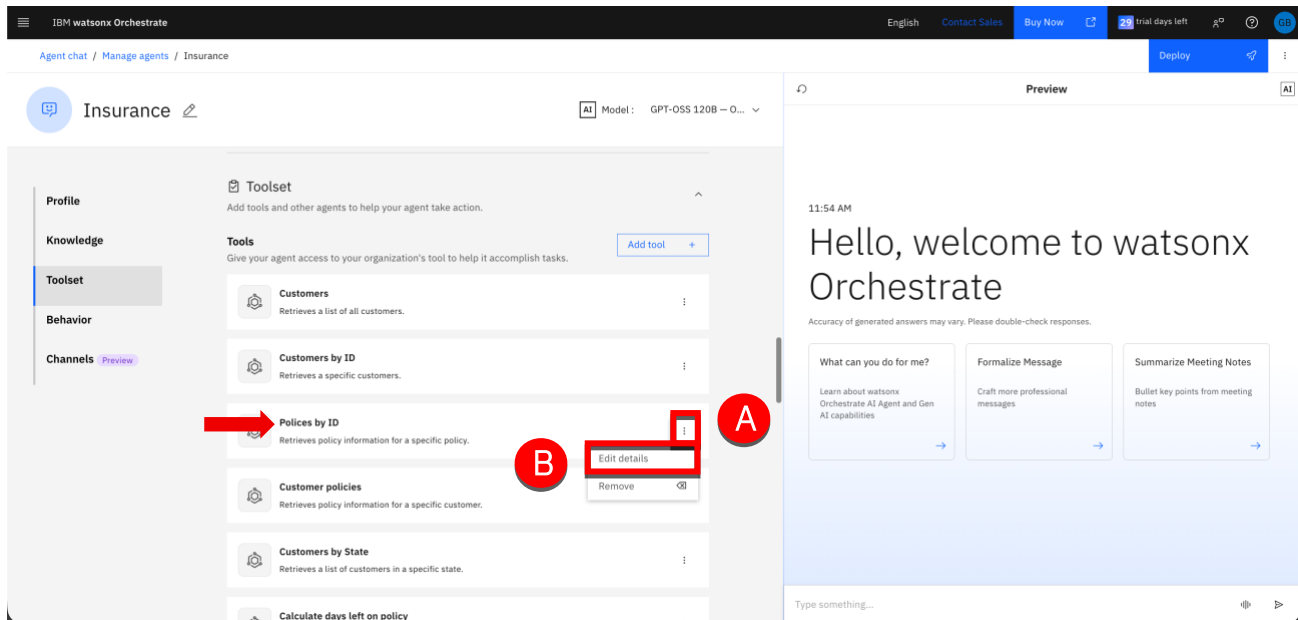


The Total premium operation performs an aggregate calculation. We want to observe how the agent performs aggregation with and without tool support, so we will import this tool in a later step.

6. The operations will be imported as tools, observe how the agent **Tools** (A) have been updated.



7. Click the three dots in the **Policies by ID** tool (A), then click **Edit details** (B).



The description from the OpenAPI specification we saw previously (A) has been extracted.

watsonx Insurance ^{1.0} ^{OAS 3.1}

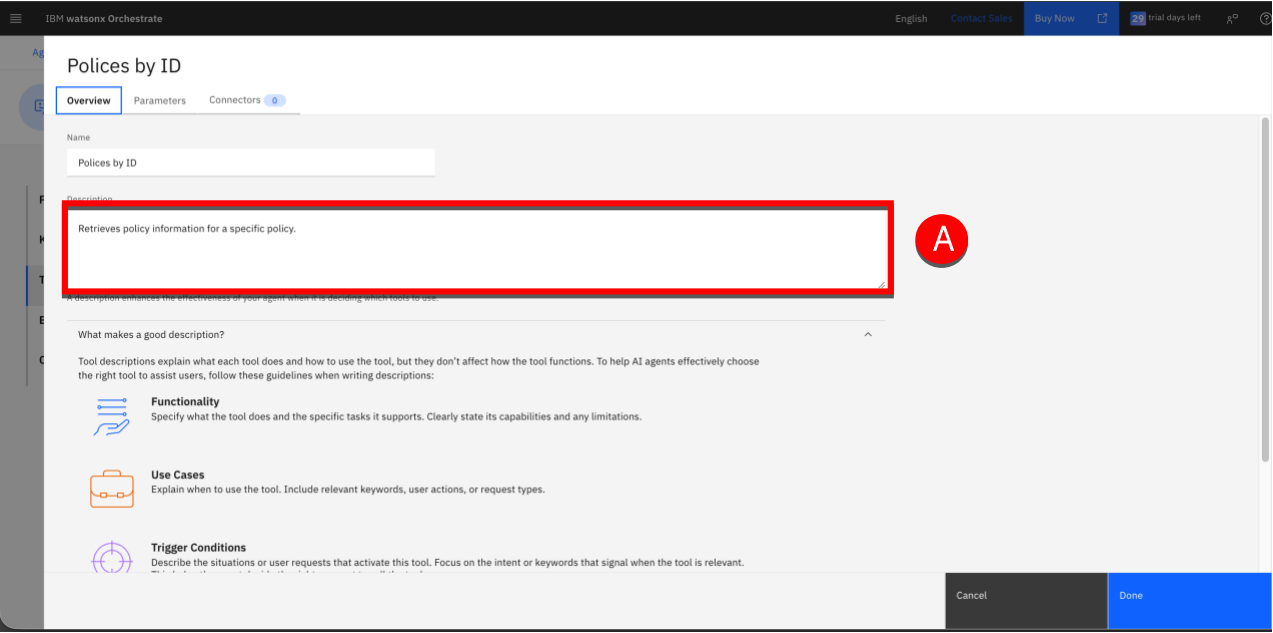
/openapi.json

APIs for watsonx Insurance services.

default

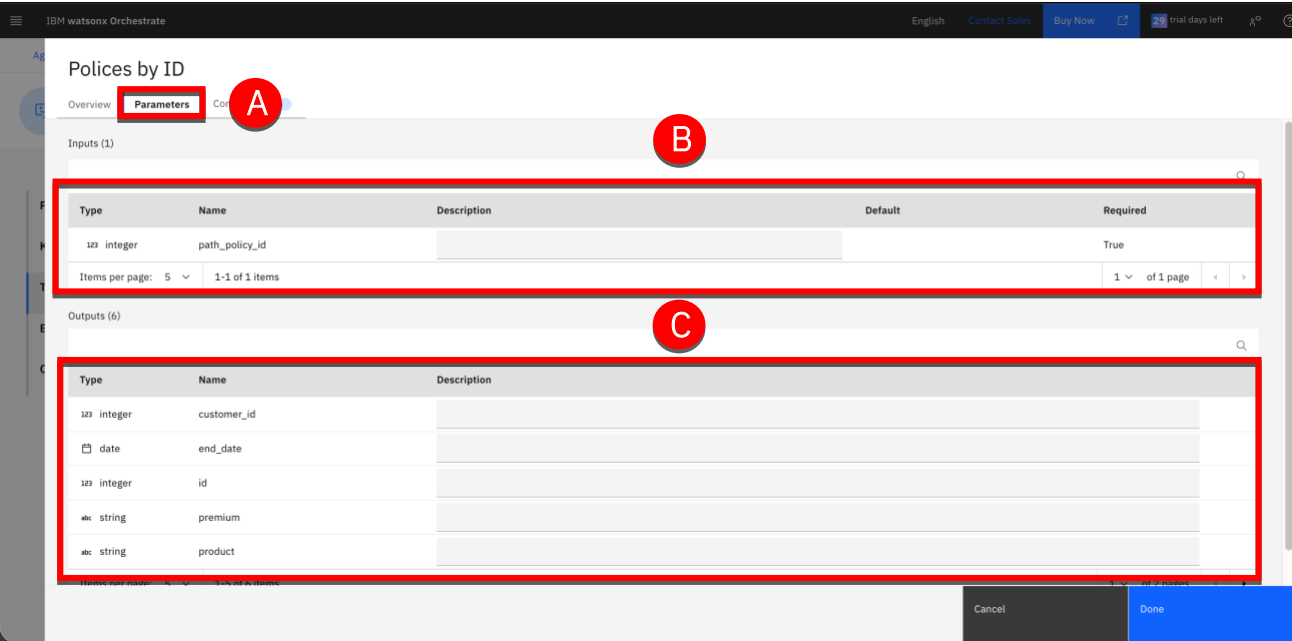
GET	/customers	Customers
GET	/customers/{customer_id}	Customers by ID
GET	/policies/{policy_id}	Policies by ID
Retrieves policy information for a specific policy.		
Parameters		
Try it out		
Name	Description	
policy_id	policy_id	
integer		
(path)		

This description has been used as the description for the tool in Orchestrate (A).



API descriptions may not be provided or lack a suitable explanation for how the API should be used. Orchestrate requires a good description as this is how the agent determines if and how a tool should be used. Avoid tools with similar descriptions and overlapping responsibilities to improve the performance of your agent.

8. Click the **Parameters** tab (A) and review the **Inputs** (B) and **Outputs** (C) of the tool.



This tool requires a policy ID and returns the details of a policy such as the product, premium, start date and end date.

Note that the premium is a currency value but is represented as a String, APIs do this to preserve numerical accuracy.

9. Click **Cancel** (A) to close this panel.

IBM watsonx Orchestrate

English Contact Sales Buy Now 14 trial days left

Policies by ID

Overview **Parameters** Connectors 0

Inputs (1)

Type	Name	Description	Default	Required
integer	path_policy_id			True

Items per page: 5 1-1 of 1 items 1 of 1 page

Outputs (6)

Type	Name	Description
integer	customer_id	
date	end_date	
integer	id	
string	premium	
string	product	

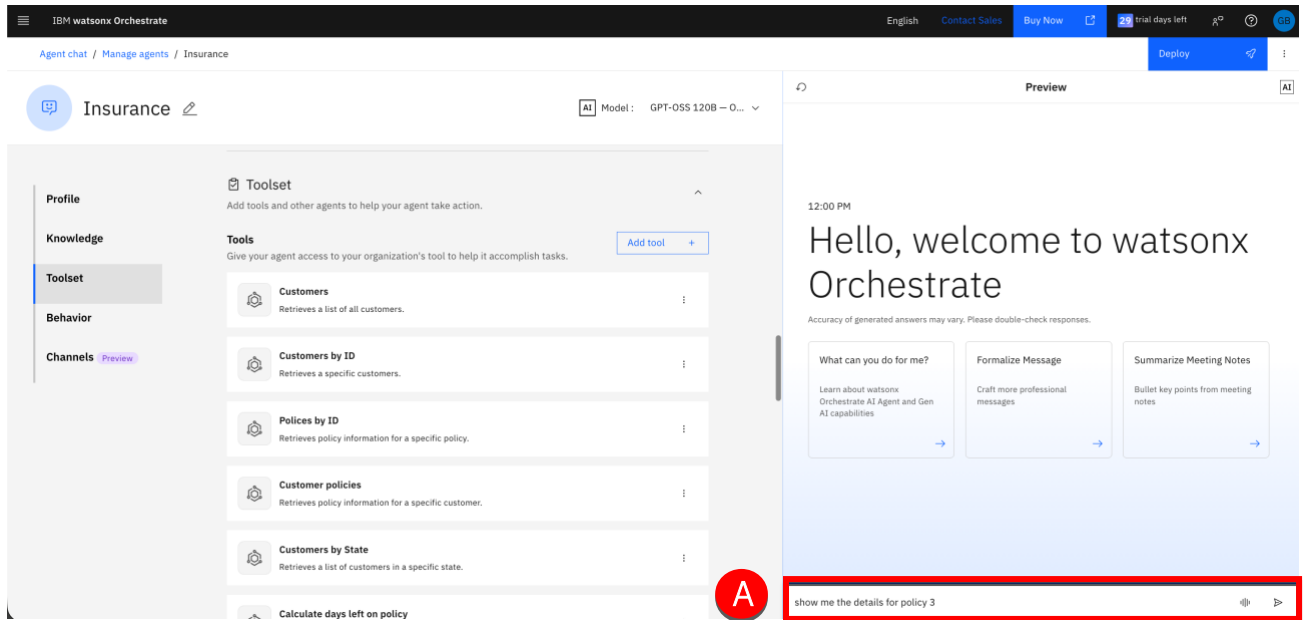
Items per page: 5 1-5 of 6 items of 2 pages

Cancel **Done**

Basic tool usage

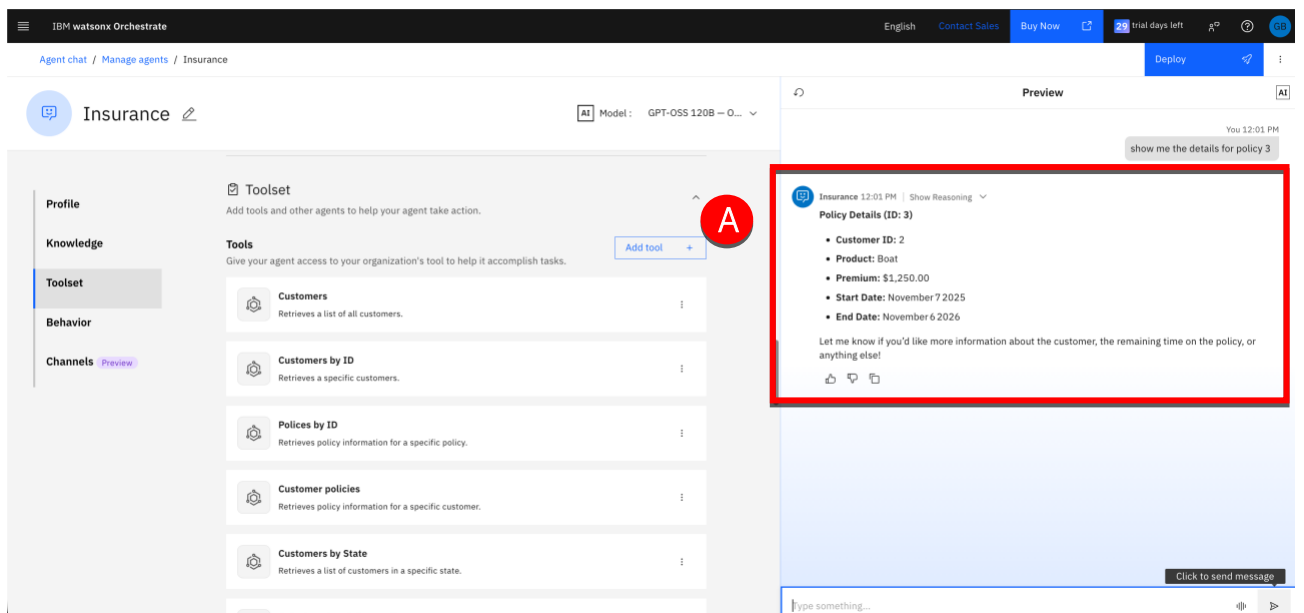
In this section we will see how the tools are used by the agent.

1. In the **Preview** panel enter “show me the details for policy 3” into the chat field (A) and press Enter.



The agent analyzes the request, determines the tool to use, and parses 3 as the target policy and uses this as a parameter when using the tool.

2. Observe how the policy details are retrieved, and a response is generated (A).



3. In the **Preview** panel enter “show me my customers” into the chat field (A) and press Enter.

The screenshot shows the IBM watsonx Orchestrate interface. On the left, the 'Insurance' agent is selected. The 'Toolset' panel lists several tools: 'Customers', 'Customers by ID', 'Policies by ID', 'Customer policies', and 'Customers by State'. The 'Preview' panel on the right shows a chat history with a message 'show me the details for policy 3' and a response 'Policy Details (ID: 3)' with fields for Customer ID, Product, Premium, Start Date, and End Date. At the bottom of the 'Preview' panel, a chat input field labeled 'A' contains the text 'show me my customers'.

4. Observe the list of customers retrieved by the agent (A).

The screenshot shows the IBM watsonx Orchestrate interface. The 'Preview' panel on the right shows a chat history with a message 'show me my customers' and a response 'Here are the customers in the system:' followed by a table of customer data. The table has columns for ID, Name, Date of Birth, Email, Address, and State. The data is as follows:

ID	Name	Date of Birth	Email	Address	State
1	John Smith	1985-06-15	john.smith@email.com	123 Main St	California
2	Sarah Johnson	1990-03-22	sarah.johnson@email.com	456 Oak Ave	Texas
3	Michael Brown	1978-11-08	michael.brown@email.com	789 Pine Rd	New York
4	Emily Davis	1995-01-30	emily.davis@email.com	321 Elm St	Florida
5	David Wilson	1982-09-12	david.wilson@email.com	654 Maple Dr	Illinois
6	Jessica Martinez	1987-04-12	jessica.martinez@email.com	789 Houston St	Texas
7	Robert Garcia	1992-08-25	robert.garcia@email.com	234 Austin Ave	Texas
8	Amanda Rodriguez	1979-12-03	amanda.rodriguez@email.com	567 Dallas	Texas

The agent has interpreted the request, used the correct tool, and displayed the results using a table.

Filtering

In this section we will use a tool to fetch a subset of customer data and see how the agent is able to filter these results even further.

1. In the **Preview** panel enter “show me my customers from Texas” into the chat field (B) and press Enter.

The screenshot displays the IBM watsonx Orchestrate interface for an 'Insurance' agent. On the left, a sidebar contains 'Profile', 'Knowledge', 'Toolset', 'Behavior', and 'Channels'. The 'Toolset' section is active, showing a list of tools: 'Customers', 'Customers by ID', 'Policies by ID', 'Customer policies', 'Customers by State', and 'Calculate days left on policy'. The 'Preview' panel on the right shows a table of customer data. A red circle with the letter 'A' highlights the chat input field at the bottom, which contains the text 'Show me my customers from Texas'.

ID	Name	DOB	Email	Address	State
6	Jessica Martinez	1987-04-12	jessica.martinez@email.com	789 Houston St	Texas
7	Robert Garcia	1992-08-25	robert.garcia@email.com	234 Austin Ave	Texas
8	Amanda Rodriguez	1979-12-03	amanda.rodriguez@email.com	567 Dallas Blvd	Texas
9	Christopher Lee	1988-01-18	christopher.lee@email.com	890 San Antonio Way	Texas
10	Michelle Thompson	1993-07-07	michelle.thompson@email.com	345 Fort Worth Dr	Texas
11	James Anderson	1984-11-29	james.anderson@email.com	678 Cedar St	Iowa
12	Lisa White	1991-05-14	lisa.white@email.com	901 Oak Hill Rd	Iowa
13	Daniel Miller	1986-09-22	daniel.miller@email.com	432 Prairie View Dr	Iowa

This agent has interpreted this request and decided to use the Customers by State tool. This is a much better approach for filtering as it avoids doing a full scan of the data and then filtering manually. Fetching all the data is not efficient at all, and the agent has limited memory, so it's essential to avoid fetching too much data.

2. Click Show Reasoning (A).

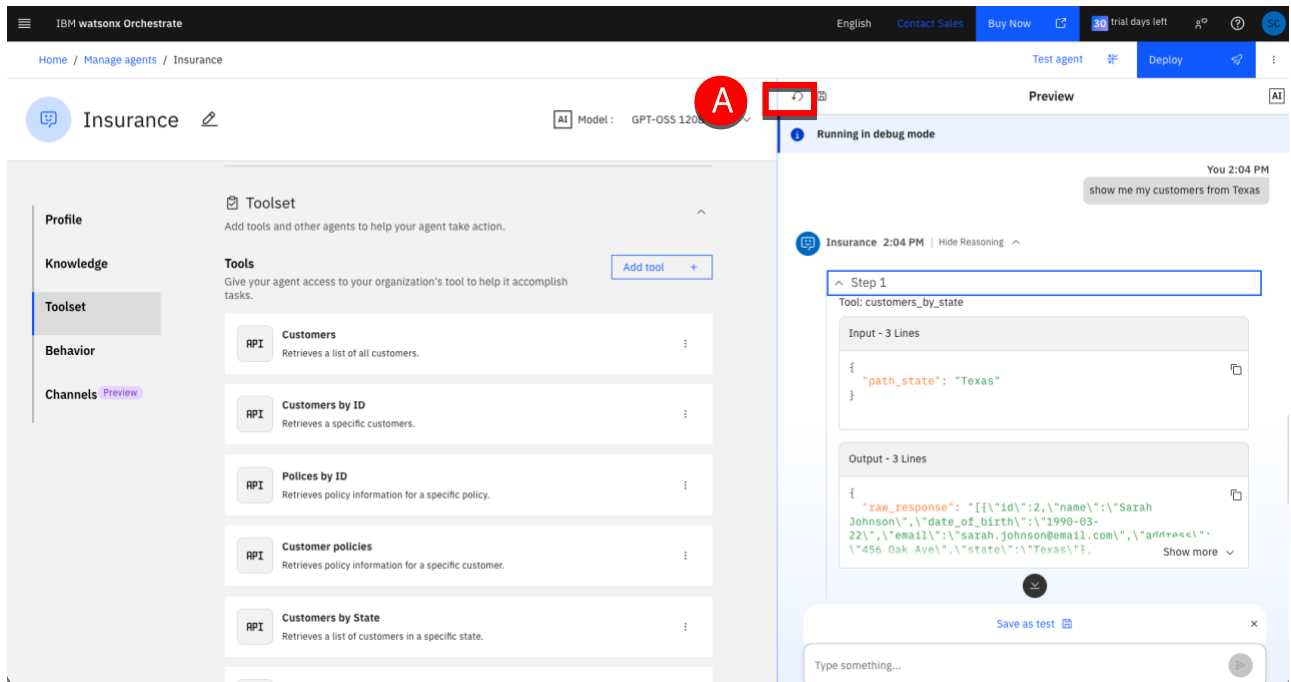
The screenshot shows the IBM watsonx Orchestrate interface. On the left, the 'Insurance' agent configuration is visible, including a 'Toolset' section with tools like 'Customers', 'Customers by ID', 'Policies by ID', 'Customer policies', 'Customers by State', and 'Calculate days left on policy'. On the right, the 'Preview' chat window shows a user request 'Show me my customers from Texas'. A red box labeled 'A' highlights the 'Show Reasoning' dropdown menu in the chat input area.

3. Click Step 1 (A) to show the tool usage.

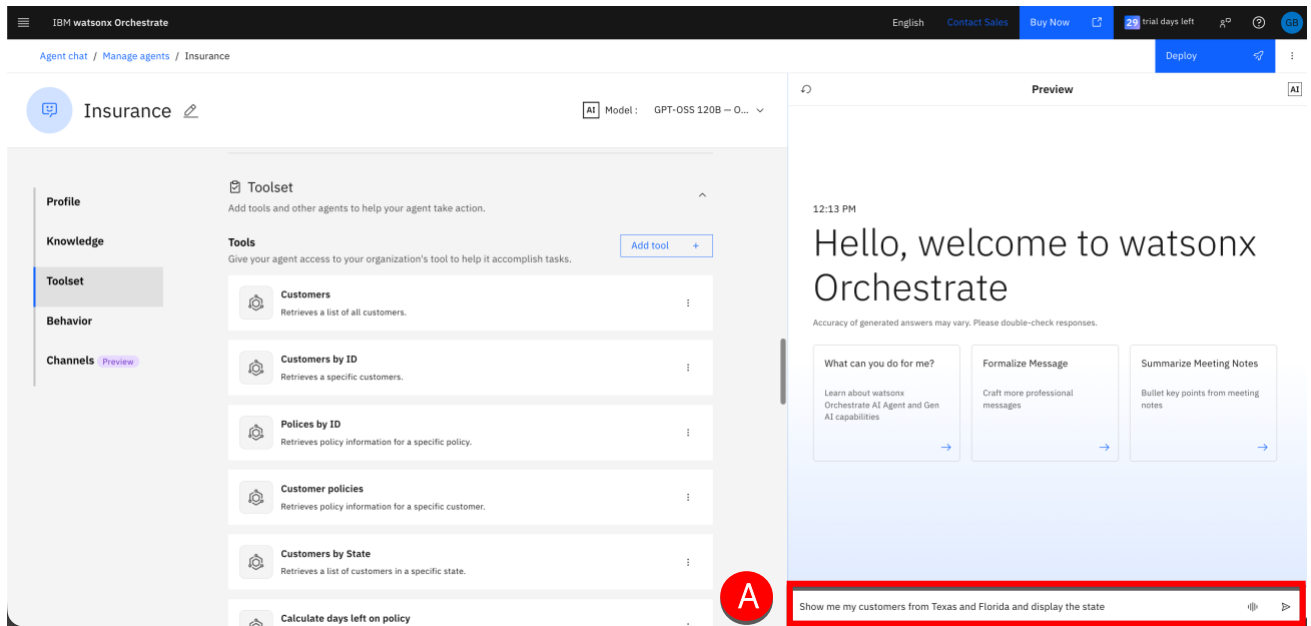
The screenshot shows the IBM watsonx Orchestrate interface. On the left, the 'Insurance' agent configuration is visible, including a 'Toolset' section with tools like 'Customers', 'Customers by ID', 'Policies by ID', 'Customer policies', 'Customers by State', and 'Calculate days left on policy'. On the right, the 'Preview' chat window shows a user request 'Show me my customers from Texas'. A red box labeled 'A' highlights the 'Step 1' dropdown menu in the chat input area.

State has been parsed from the request and passed to the tool called customers by state. The raw output from the tool which is used to generate the results table is shown in the output.

4. Click the reset icon (A) to reset the chat and clear the agent's memory.



5. In the **Preview** panel enter “Show me my customers from Texas and Florida and display the state” into the chat field (A) and press Enter.



The request has been analyzed and Orchestrate has determined that two tool calls are required to satisfy the request. The results from both tool calls are then combined to generate the response.

Insurance Model: GPT-OSS 120B — 0...

Toolset
Add tools and other agents to help your agent take action.

Tools
Give your agent access to your organization's tool to help it accomplish tasks.

- Customers**
Retrieves a list of all customers.
- Customers by ID**
Retrieves a specific customers.
- Policies by ID**
Retrieves policy information for a specific policy.
- Customer policies**
Retrieves policy information for a specific customer.
- Customers by State**
Retrieves a list of customers in a specific state.
- Calculate days left on policy**

Preview

You 12:14 PM
Show me my customers from Texas and Florida and display the state

Insurance 12:14 PM | Show Reasoning

Customers in Texas

ID	Name	Email	State
2	Sarah Johnson	sarah.johnson@email.com	Texas
6	Jessica Martinez	jessica.martinez@email.com	Texas
7	Robert Garcia	robert.garcia@email.com	Texas
8	Amanda Rodriguez	amanda.rodriguez@email.com	Texas
9	Christopher Lee	christopher.lee@email.com	Texas
10	Michelle Thompson	michelle.thompson@email.com	Texas

Customers in Florida

ID	Name	Email	State
4	Emily Davis	emily.davis@email.com	Florida

Type something...

- In the **Preview** panel enter “just show me my customers from florida” into the chat field (B) and press Enter. Note the use of lower-case “florida”.

Insurance Model: GPT-OSS 120B — 0...

Toolset
Add tools and other agents to help your agent take action.

Tools
Give your agent access to your organization's tool to help it accomplish tasks.

- Customers**
Retrieves a list of all customers.
- Customers by ID**
Retrieves a specific customers.
- Policies by ID**
Retrieves policy information for a specific policy.
- Customer policies**
Retrieves policy information for a specific customer.
- Customers by State**
Retrieves a list of customers in a specific state.
- Calculate days left on policy**

Preview

You 12:14 PM
Show me my customers from Texas and Florida and display the state

Insurance 12:14 PM | Show Reasoning

Customers in Texas

ID	Name	Email	State
2	Sarah Johnson	sarah.johnson@email.com	Texas
6	Jessica Martinez	jessica.martinez@email.com	Texas
7	Robert Garcia	robert.garcia@email.com	Texas
8	Amanda Rodriguez	amanda.rodriguez@email.com	Texas
9	Christopher Lee	christopher.lee@email.com	Texas
10	Michelle Thompson	michelle.thompson@email.com	Texas

Customers in Florida

ID	Name	Email	State
4	Emily Davis	emily.davis@email.com	Florida

just show me my customers from florida

7. Observe the response (A) showing customers from Florida.

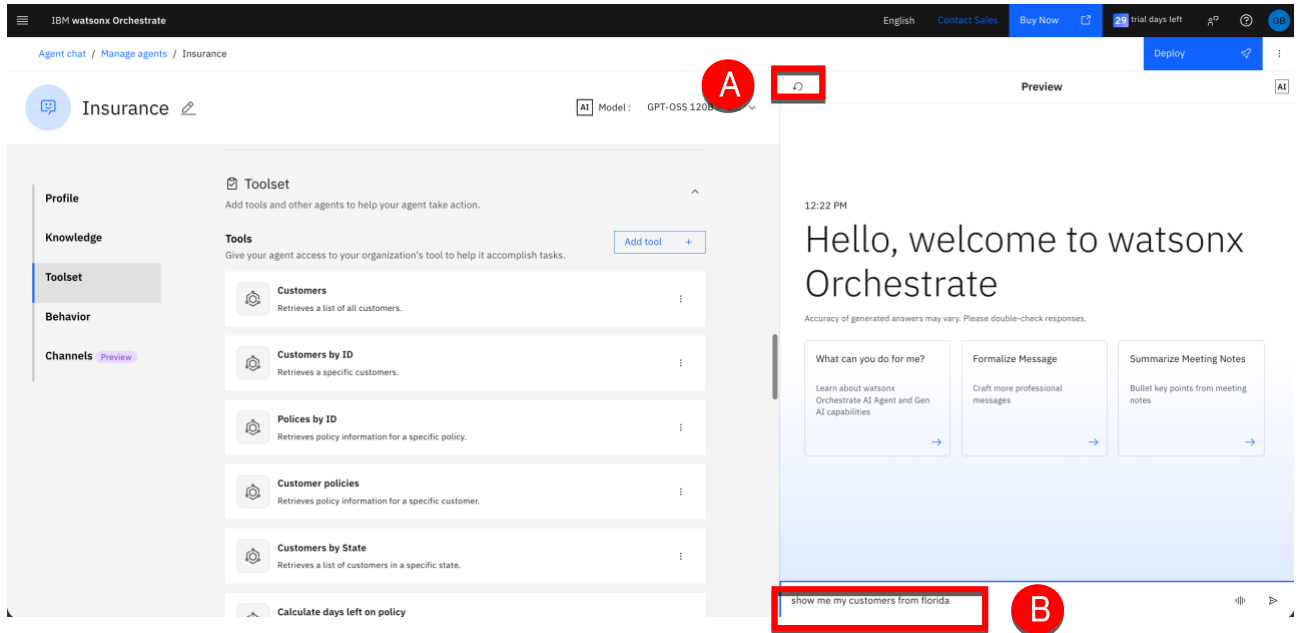
The screenshot shows the IBM watsonx Orchestrate interface. On the left, there's a sidebar with 'Insurance' selected. The main area shows a chat window with a user input 'just show me my customers from florida' and an AI response (A) displaying a table of customer data filtered by Florida. The table has columns: id, name, date_of_birth, email, address, and state. The response is highlighted with a red box and a red circle with 'A'.

id	name	date_of_birth	email	address	state
4	Emily Davis	1995-01-30	emily.davis@email.com	321 Elm St	Florida

When the agent responds there is no tool information as the agent has determined that it has sufficient information already available in its chat history.

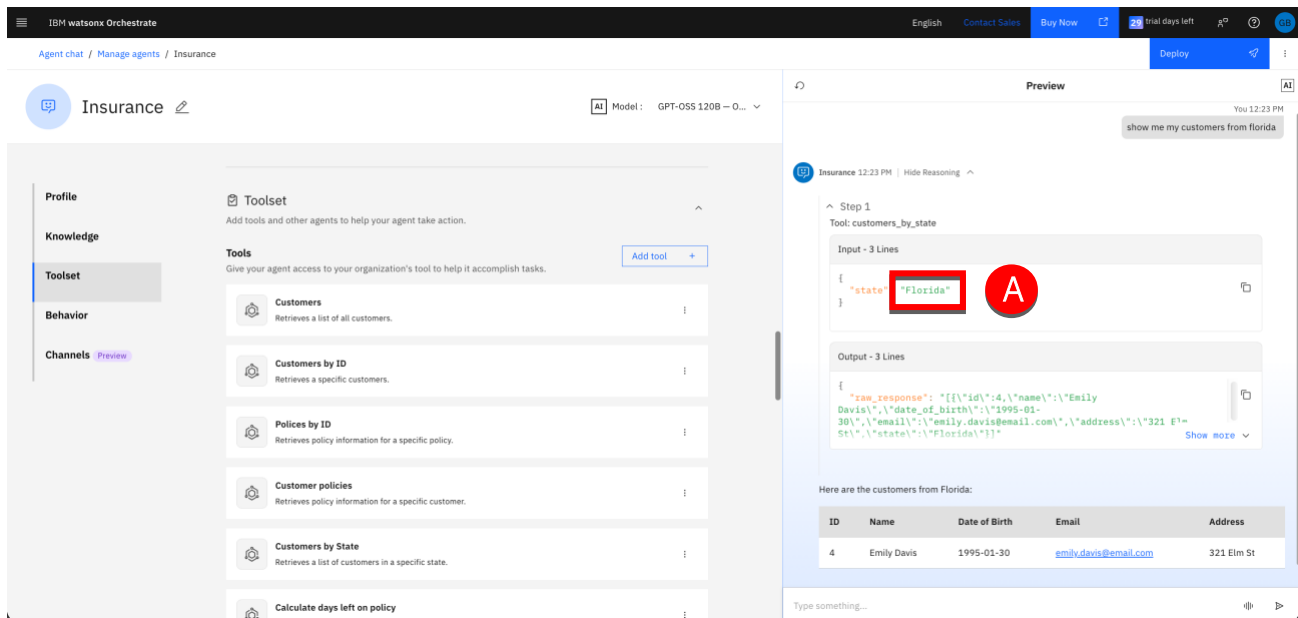
The agent has filtered for customers from florida and done so, even though lower-case florida was specified. However, the tool is case-sensitive highlighting how the agent can provide different results based on conversation history.

- Click the reset icon (A) to clear the agent memory, then enter “show me my customers from florida” into the chat field (B) and press Enter.



Using the tool with florida as a parameter may, or may not retrieve any records, it depends on the agent. The API is case sensitive and will return zero records for “florida”.

In the example below, “florida” was provided as a parameter, but the agent has corrected this and submitted Florida as a parameter (A).



This behavior is not guaranteed and can result in inconsistent results. In reality, you would need ensure the tool was able to work in a case-insensitive way and able to cope with abbreviations such as TX for Texas.

Aggregation

In this section we will ask the agent to perform calculations, and see why complex calculations should be performed using tools.

1. Click the reset icon (A) to clear the agent memory, then enter “show me the policies for customer 2” into the chat field (B) and press Enter.

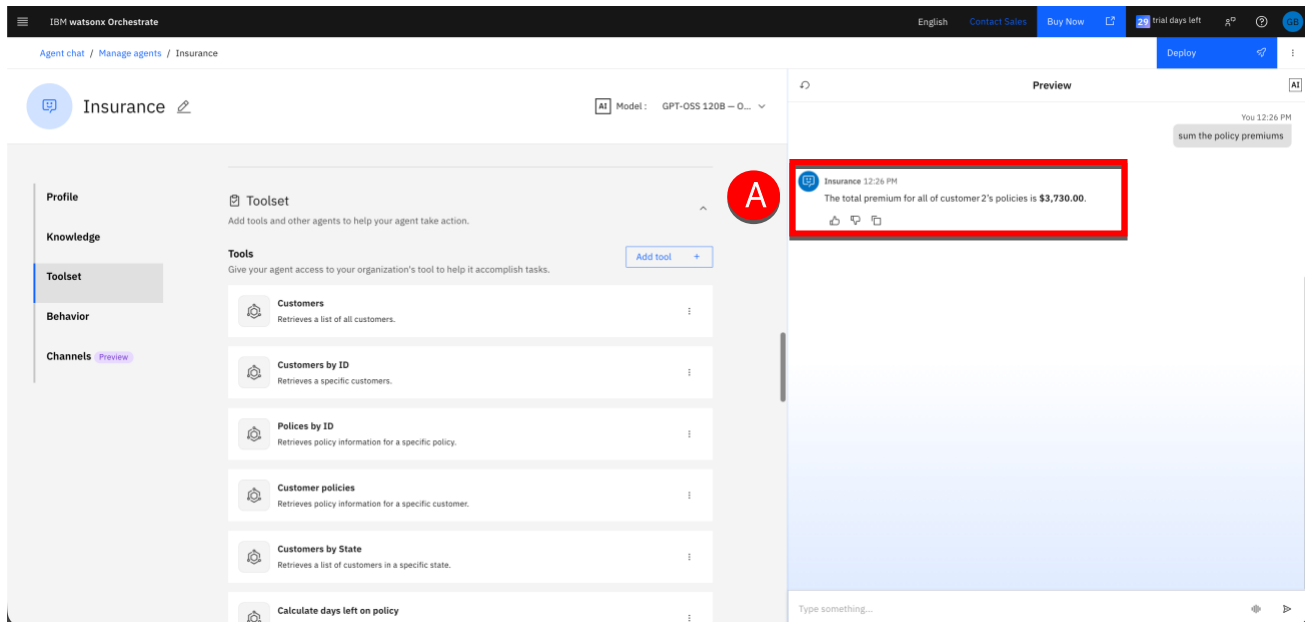
The screenshot shows the IBM watsonx Orchestrate interface for an 'Insurance' agent. The left sidebar contains a 'Toolset' section with a list of tools: 'Customers', 'Customers by ID', 'Policies by ID', 'Customer policies', 'Customers by State', and 'Calculate days left on policy'. The main chat area on the right shows a welcome message and a chat input field (B) containing the text 'show me the policies for customer 2'. A red box (A) highlights the reset icon in the top right corner of the chat area.

2. Enter “sum the policy premiums” into the chat field (A) and press Enter.

The screenshot shows the IBM watsonx Orchestrate interface for an 'Insurance' agent. The chat area on the right shows a chat history with the previous query 'show me the policies for customer 2' and the resulting table of policies. The chat input field (A) contains the text 'sum the policy premiums'. The table of policies is as follows:

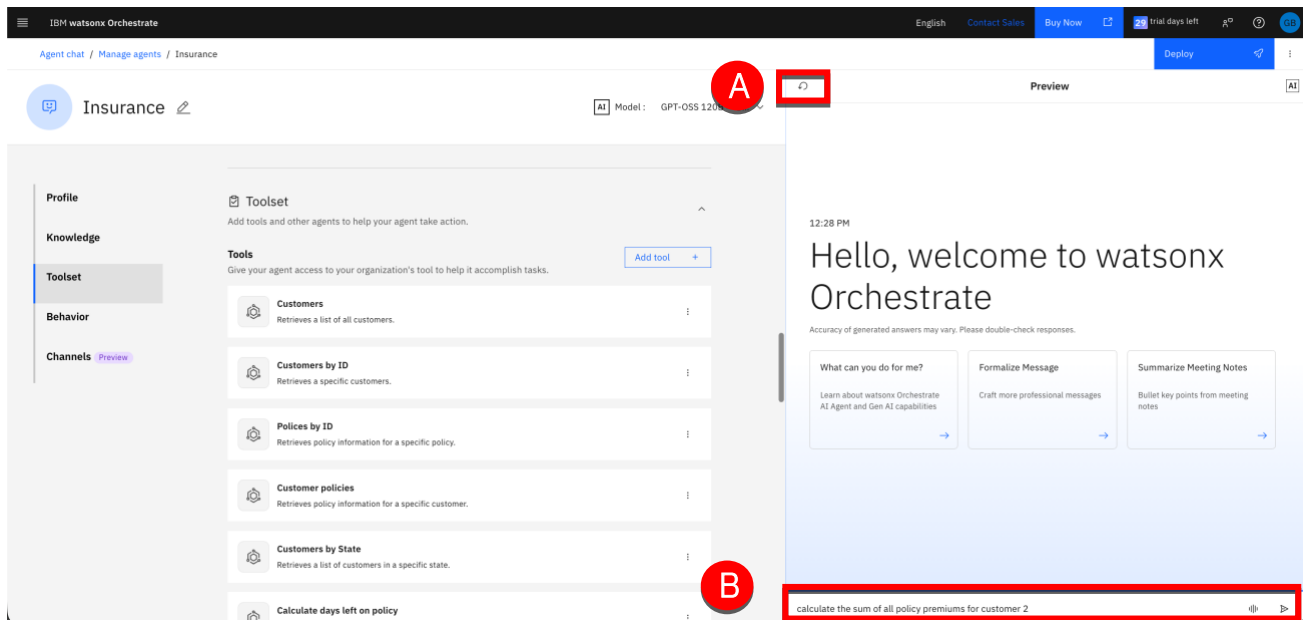
Policy ID	Product	Start Date	End Date	Premium
3	Boat	2025-11-07	2026-11-06	\$1,250.00
4	RV	2025-10-08	2026-10-07	\$2,100.00
23	Pet	2025-01-27	2026-01-26	\$380.00

3. Observe the calculation result (A).



The agent has summed the policy premiums correctly. It had the premium information readily at hand in the conversation history.

4. Click the reset icon (A) to clear the agent memory, then enter “calculate the sum of all policy premiums for customer 2” into the chat field (B) and press Enter.



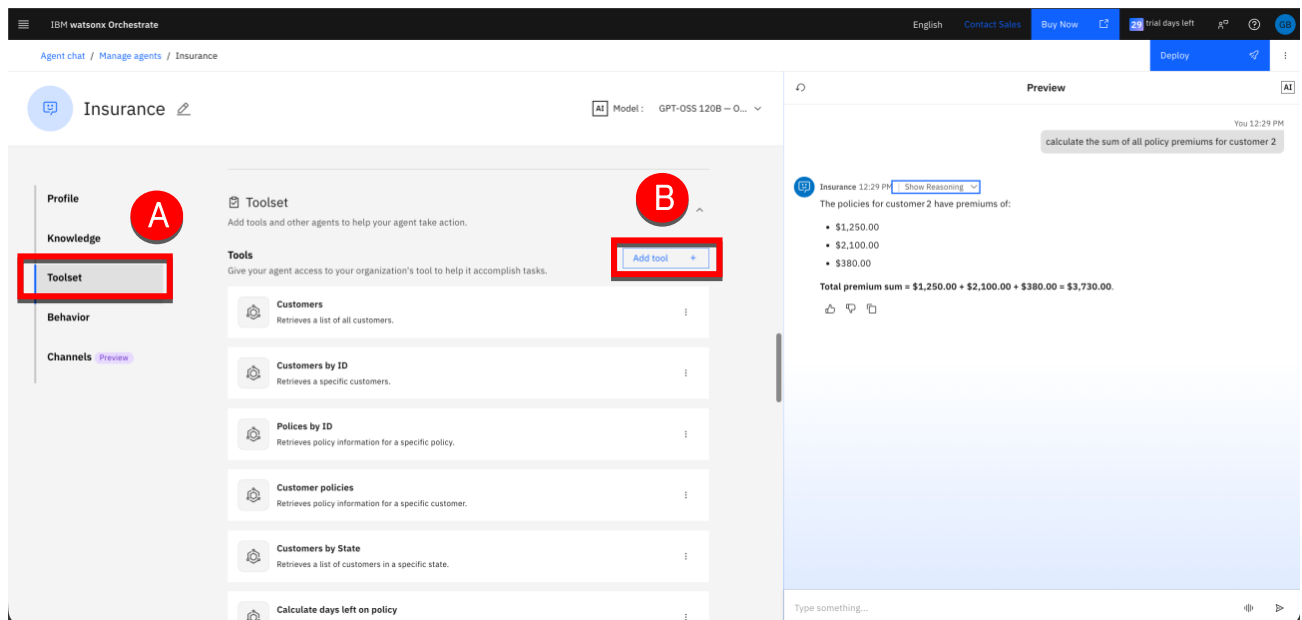
5. Observe the calculation result (A).

The screenshot displays the IBM watsonx Orchestrate interface. On the left, a sidebar contains navigation links: Profile, Knowledge, Toolset (selected), Behavior, and Channels. The main area is titled 'Insurance' and shows a 'Toolset' section with various tools like 'Customers', 'Customers by ID', 'Policies by ID', 'Customer policies', 'Customers by State', and 'Calculate days left on policy'. On the right, a 'Preview' window shows a chat conversation. The user's message is 'calculate the sum of all policy premiums for customer 2'. The AI's response, timestamped 'Insurance 12:29 PM', lists three premium amounts: \$1,250.00, \$2,100.00, and \$380.00, and then calculates the total: 'Total premium sum = \$1,250.00 + \$2,100.00 + \$380.00 = \$3,730.00'. A red box highlights this AI response, and a red circle with the letter 'A' is placed next to it.

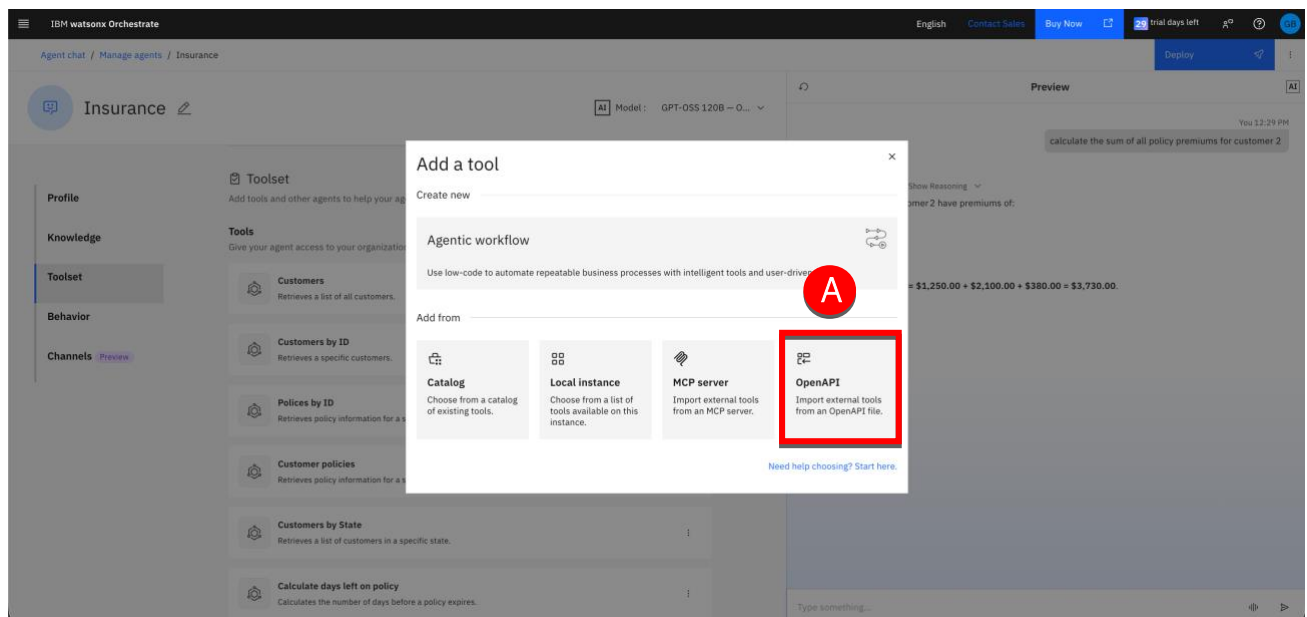
The request we've entered is like the previous one, but here the agent did not have the policy information in its conversational history, so it called a tool to fetch the policies and used the LLM to perform the calculation. As a result, the probability of an accurate result is not 100%, especially when performing more complex calculations. The latest models perform well, but the observed results from some models have included \$2730, \$2830, or \$3730 (the correct result).

The LLM is a text generator, not a calculator, and it's not an insurance specialist either. The policy data could contain multiple currency attributes (premium, excess, tax) making it very easy for the LLM generating the response to make an error. We could try using specific agent instructions, but this still isn't 100% accurate and reliable, the correct approach for calculations is to use a tool.

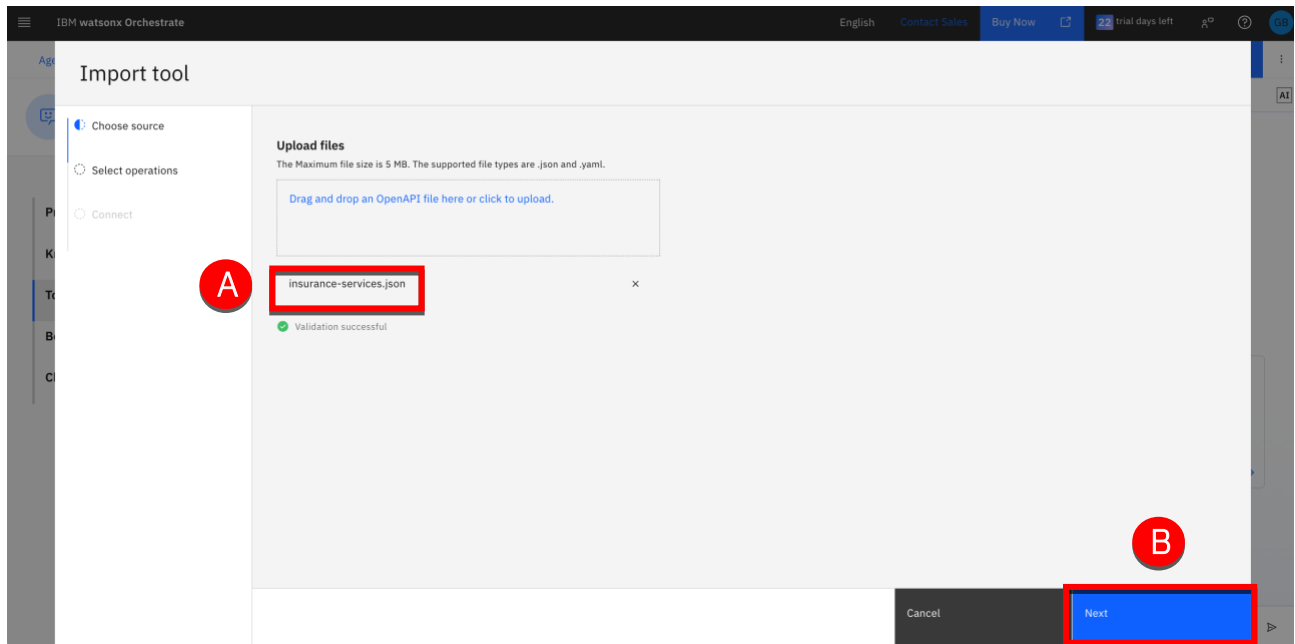
6. Click **Toolset** (A), then **Add tool** (B).



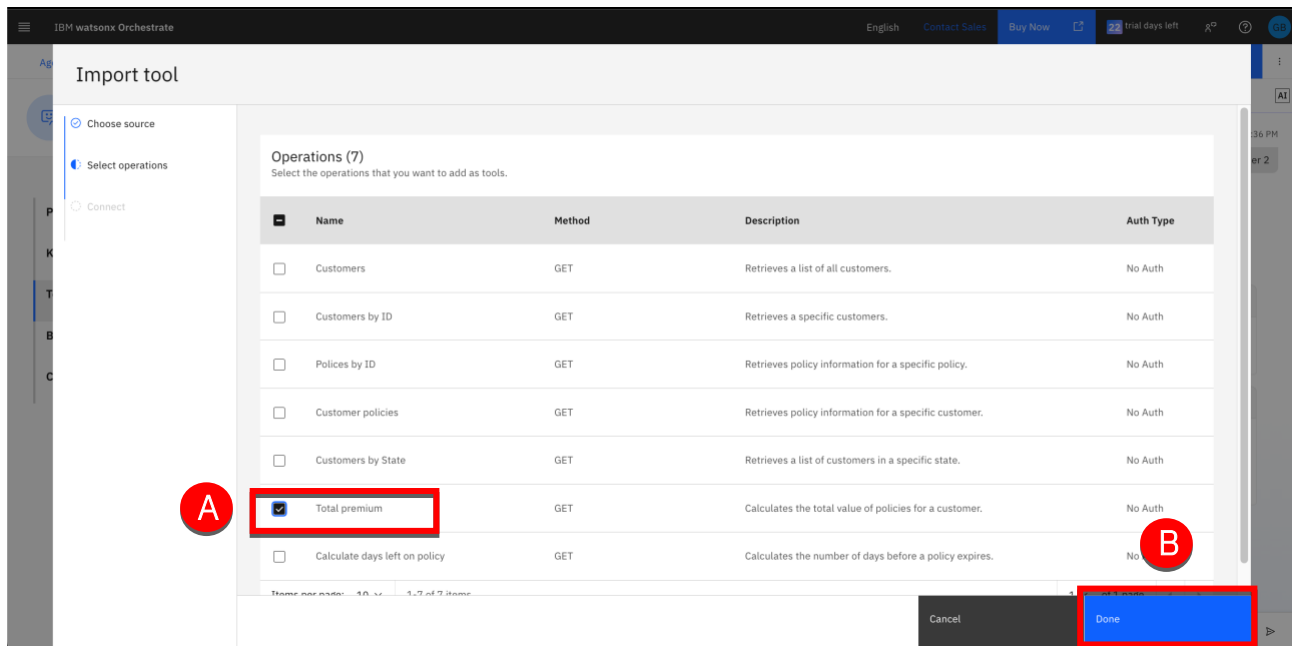
7. Click **OpenAPI** (A).



8. Upload or drag and drop the **insurance-service.json** file (A), then click **Next** (B).

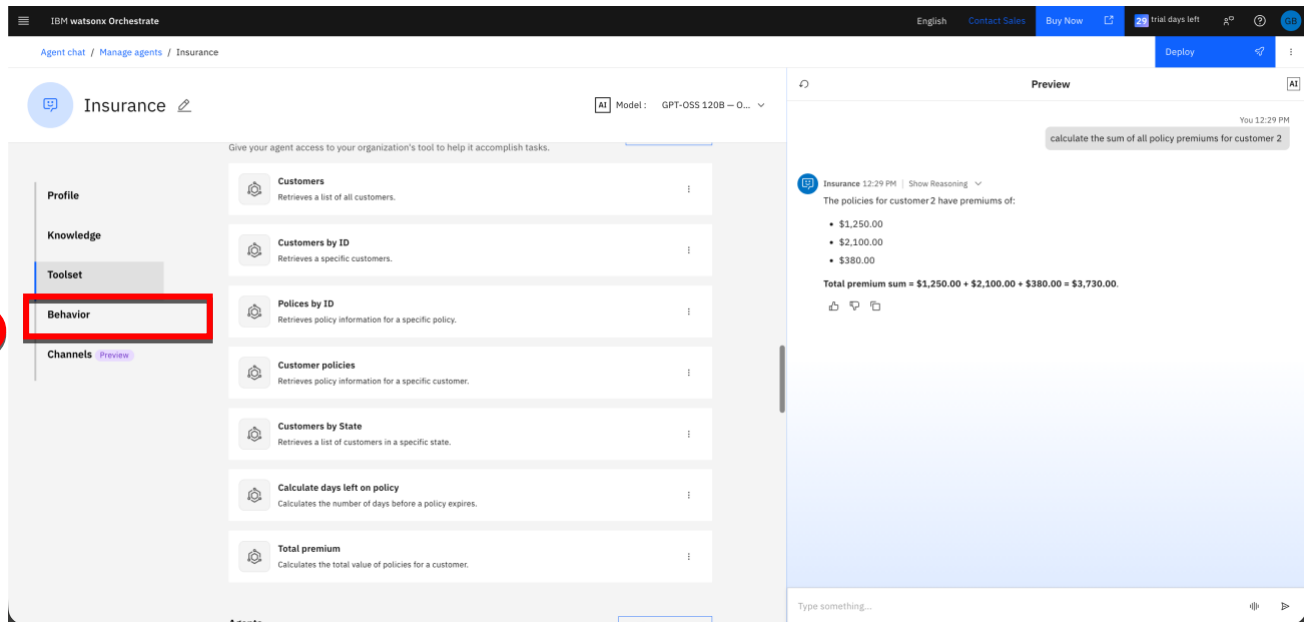


9. Select **Total premium** (A), then click **Done** (B)



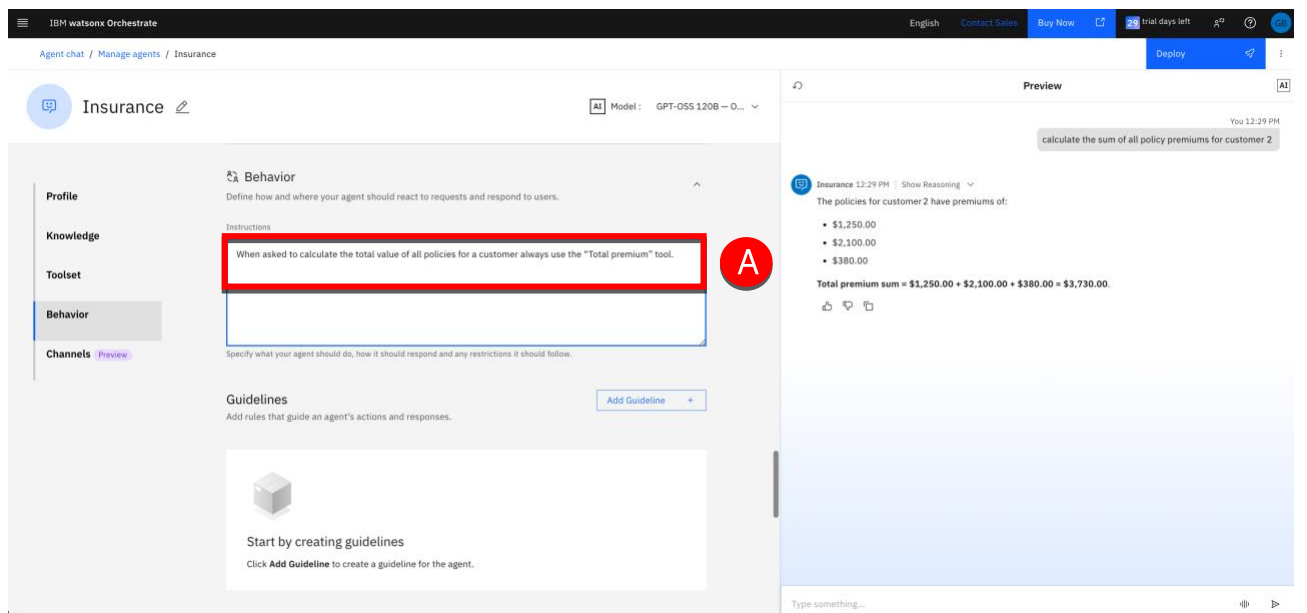
To ensure the agent uses this new tool for the calculation we can also add instructions to the agent.

10. Click **Behavior** (A).

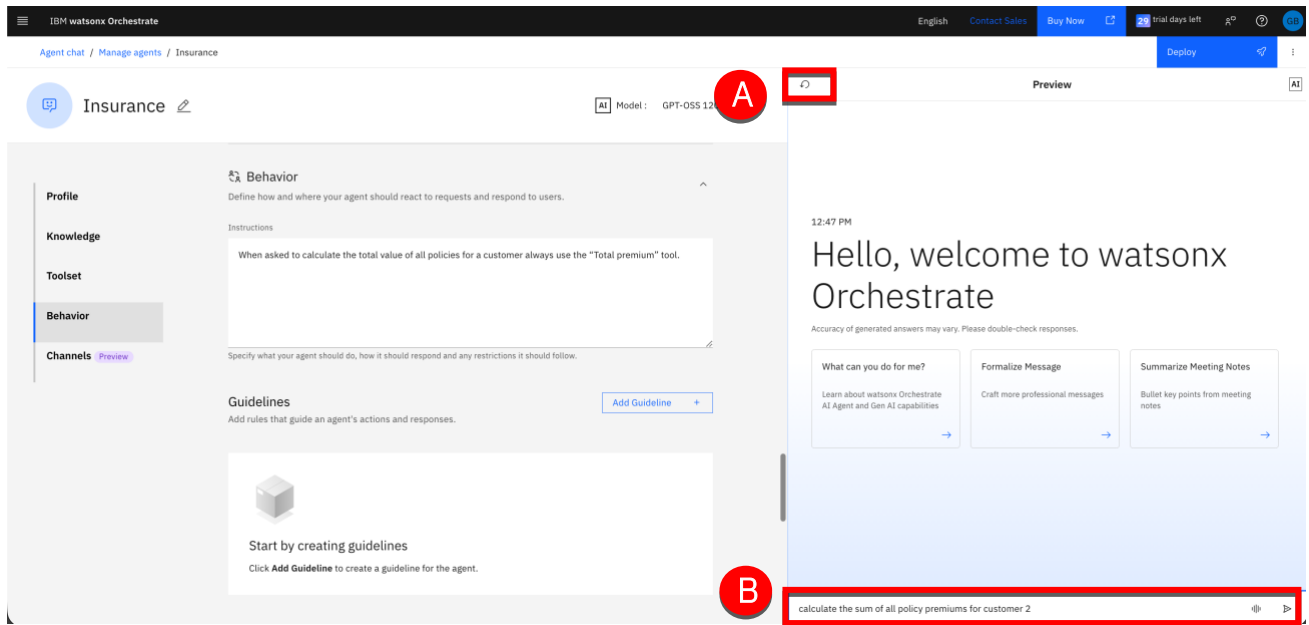


11. Add the following text into the **Instructions** field (A).

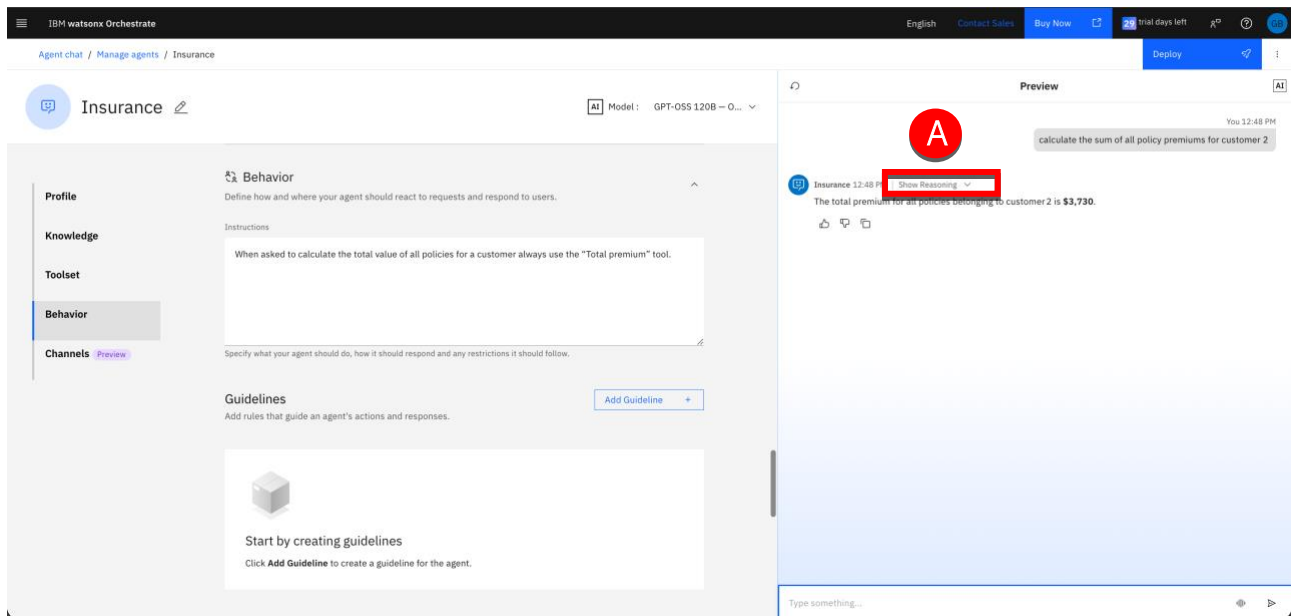
When asked to calculate the total value of all policies for a customer always use the "Total premium" tool.



12. Click the reset icon (A) to clear the agent memory, then enter “calculate the sum of all policy premiums for customer 2” into the chat field (B) and press Enter.



13. Click Show Reasoning (A).



14. Click **Step 1 (A)** to display the tool information.

The screenshot displays the IBM watsonx Orchestrate interface. On the left, the 'Insurance' agent configuration is visible, with the 'Behavior' tab selected. The 'Instructions' section contains the text: 'When asked to calculate the total value of all policies for a customer always use the "Total premium" tool.' The 'Guidelines' section is empty, with a button to 'Add Guideline'. On the right, the 'Preview' section shows the agent's reasoning process. A red box highlights 'Step 1' in the reasoning chain. The input is a JSON object: `{ "customer_id": 2 }`. The output is a JSON object: `{ "customer_id": 2, "total_premium": 3730 }`. The final result displayed is: 'The total premium for all policies belonging to customer 2 is \$3,730.'

The new tool has been used, and more importantly the result is based on the result from a tool, instead of relying upon the LLM.

Context

In this section we will see how conversational context can influence the results provided by the agent.

1. In the **Preview** panel click the reset icon (A), then enter “show me my customers from Texas” into the chat field (B) and press Enter.

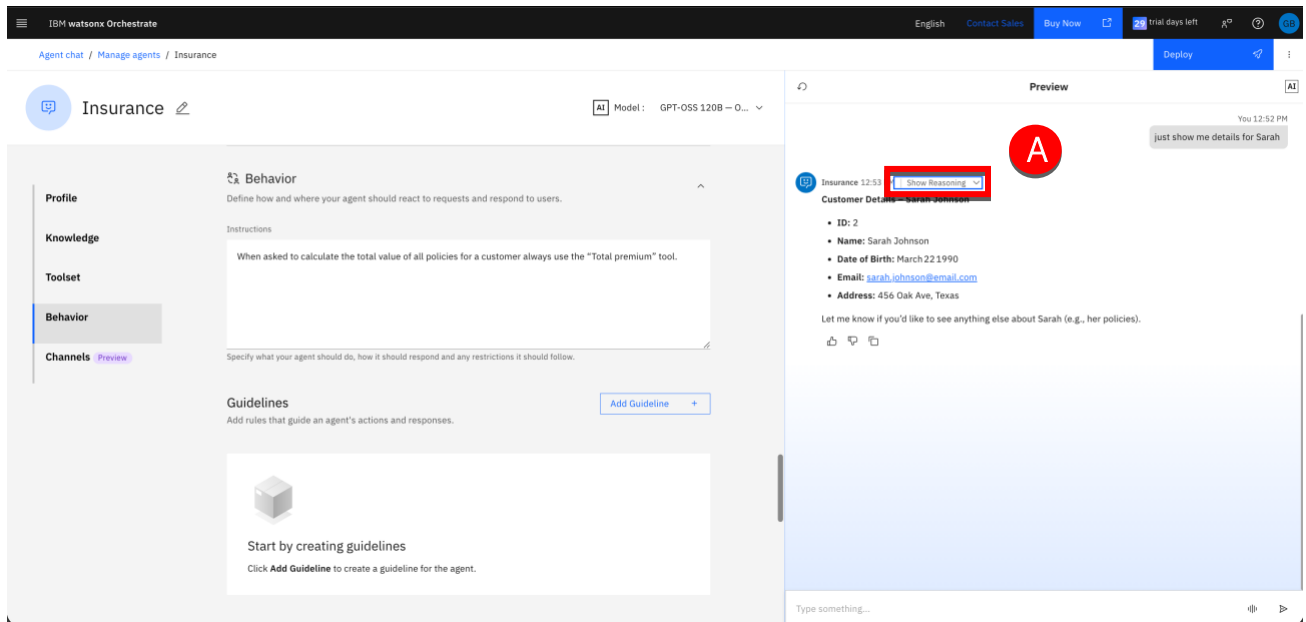
The screenshot shows the IBM watsonx Orchestrate interface. The left sidebar contains navigation links: Profile, Knowledge, Toolset, Behavior, and Channels. The main area is divided into two panels. The left panel is titled 'Insurance' and contains a 'Behavior' section with instructions and guidelines. The right panel is titled 'Preview' and shows a chat interface. In the 'Preview' panel, a red box labeled 'A' highlights the reset icon in the top right corner. Another red box labeled 'B' highlights the chat input field at the bottom right, which contains the text 'show me my customers from Texas'.

2. Enter “just show me details for Sarah” into the chat field (A) and press Enter.

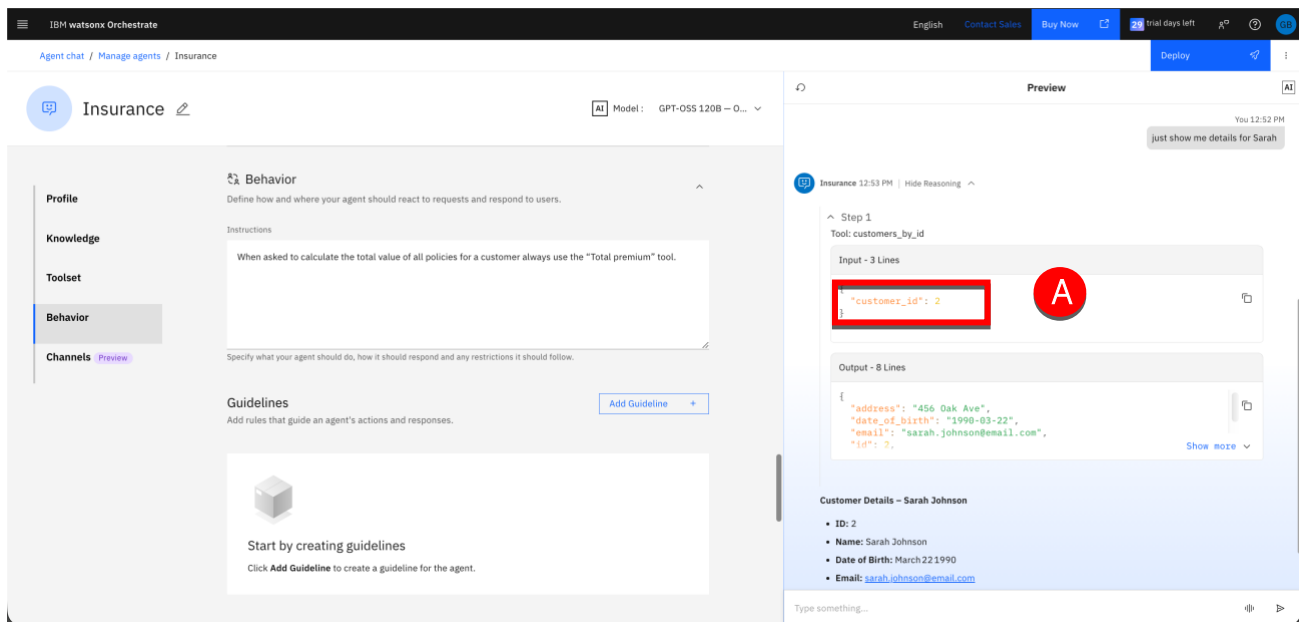
The screenshot shows the IBM watsonx Orchestrate interface. The left sidebar contains navigation links: Profile, Knowledge, Toolset, Behavior, and Channels. The main area is divided into two panels. The left panel is titled 'Insurance' and contains a 'Behavior' section with instructions and guidelines. The right panel is titled 'Preview' and shows a chat interface. In the 'Preview' panel, a red box labeled 'A' highlights the chat input field at the bottom right, which contains the text 'just show me details for Sarah'.

ID	Name	Date of Birth	Email	Address
2	Sarah Johnson	1990-03-22	sarah.johnson@email.com	456 Oak Ave
6	Jessica Martinez	1987-04-12	jessica.martinez@email.com	789 Houston St
7	Robert Garcia	1992-08-25	robert.garcia@email.com	234 Austin Ave
8	Amanda Rodriguez	1979-12-03	amanda.rodriguez@email.com	567 Dallas Blvd
9	Christopher Lee	1988-01-18	christopher.lee@email.com	890 San Antonio Way
10	Michelle Thompson	1993-07-07	michelle.thompson@email.com	345 Fort Worth Dr

3. Click Show Reasoning (A).



4. Observe that the agent has correctly determined the ID for the customer called Sarah (A).



- Enter “Fetch Sarah’s policies, show me any policies that end within the next 30 days” into the chat field (A) and press Enter.

The screenshot shows the IBM watsonx Orchestrate interface. On the left, the 'Insurance' agent configuration is visible, with the 'Behavior' tab selected. The 'Behavior' section contains instructions and guidelines. On the right, the 'Preview' chat window shows a conversation. The user's input is "Fetch Sarah's policies, show me any policies that end within the next 30 days", which is highlighted in a red box and labeled with a red circle 'A'. The chat history shows a previous message from the user: "just show me details for Sarah". The chat response from the agent is displayed below the input field.

IBM watsonx Orchestrate

Agent chat / Manage agents / Insurance

Insurance

Model: GPT-OS5 120B - 0...

Profile

Knowledge

Toolset

Behavior

Channels Preview

Behavior

Define how and where your agent should react to requests and respond to users.

Instructions

When asked to calculate the total value of all policies for a customer always use the "Total premium" tool.

Guidelines

Add rules that guide an agent's actions and responses.

Add Guideline +

Start by creating guidelines

Click Add Guideline to create a guideline for the agent.

Preview

You 12:52 PM

just show me details for Sarah

Insurance 12:53 PM Hide Reasoning

Step 1

Tool: customers_by_id

Input - 3 Lines

```
{
  "customer_id": 2
}
```

Output - 8 Lines

```
{
  "address": "456 Oak Ave",
  "date_of_birth": "1990-03-22",
  "email": "sarah.johnson@email.com",
  "id": 2
}
```

Customer Details - Sarah Johnson

- ID: 2
- Name: Sarah Johnson
- Date of Birth: March 22 1990
- Email: sarah.johnson@email.com

Fetch Sarah's policies, show me any policies that end within the next 30 days

- Click Show Reasoning (A).

The screenshot shows the IBM watsonx Orchestrate interface. On the left, the 'Insurance' agent configuration is visible, with the 'Behavior' tab selected. The 'Behavior' section contains instructions and guidelines. On the right, the 'Preview' chat window shows a conversation. The user's input is "Fetch Sarah's policies, show me any policies that end within the next 30 days", which is highlighted in a red box and labeled with a red circle 'A'. The chat history shows a previous message from the user: "just show me details for Sarah". The chat response from the agent is displayed below the input field, showing a table of policies and a summary of other policies.

IBM watsonx Orchestrate

Agent chat / Manage agents / Insurance

Insurance

Model: GPT-OS5 120B - 0...

Profile

Knowledge

Toolset

Behavior

Channels Preview

Behavior

Define how and where your agent should react to requests and respond to users.

Instructions

When asked to calculate the total value of all policies for a customer always use the "Total premium" tool.

Guidelines

Add rules that guide an agent's actions and responses.

Add Guideline +

Start by creating guidelines

Click Add Guideline to create a guideline for the agent.

Preview

You 12:56 PM

Fetch Sarah's policies, show me any policies that end within the next 30 days

Insurance 12:56 PM Show Reasoning

Sarah Johnson

Policy ID	Product	Premium	End Date	Days Until Expiration
23	Pet	\$380.00	2026-01-26	20 days

All other policies for Sarah (IDs 3 and 4) have more than 30 days remaining before they expire. Let me know if you'd like details on those policies or need anything else!

Type something...

7. Observe that four tool calls were required (A).

The screenshot shows the IBM watsonx Orchestrate interface. On the left, the 'Insurance' agent configuration is visible, with the 'Behavior' tab selected. The 'Instructions' section contains the text: 'When asked to calculate the total value of all policies for a customer always use the "Total premium" tool.' The 'Preview' panel on the right shows a conversation history. A user message asks to 'Fetch Sarah's policies, show me any policies that end within the next 30 days'. The agent's response, timestamped 12:56 PM, shows a list of policies for Sarah (ID 23) and a summary of other policies. A red box highlights the 'Step 1' through 'Step 4' tool calls, and a red circle with the letter 'A' is placed next to it.

IBM watsonx Orchestrate

Agent chat / Manage agents / Insurance

Model: GPT-OSS 120B - 0...

Preview

You 12:56 PM

Fetch Sarah's policies, show me any policies that end within the next 30 days

Insurance 12:56 PM

- Step 1
- Step 2
- Step 3
- Step 4

Policy ID Product Premium End Date Days Until Expiration

Policy ID	Product	Premium	End Date	Days Until Expiration
23	Pet	\$380.00	2026-01-26	20 days

All other policies for Sarah (IDs 3 and 4) have more than 30 days remaining before they expire. Let me know if you'd like details on those policies or need anything else!

The agent has been able to respond correctly because of context. Sarah's details, including the customer ID were available in the conversation history. Four tool calls were required to process the request, the first fetched all three of Sarah's policies, then a further tool call was made for each policy to calculate the days remaining.

8. Click **Step 1** (A) to expand the tool information.

The screenshot shows the IBM watsonx Orchestrate interface. On the left, the 'Insurance' agent configuration is visible, with the 'Profile' tab selected. The 'Description' section contains the text: 'This agent retrieves information about customers and their policies.' The 'Preview' panel on the right shows a conversation history. A user message asks to 'Fetch Sarah's policies, show me any policies that end within the next 30 days'. The agent's response, timestamped 04:53 PM, shows a list of policies for Sarah (ID 23) and a summary of other policies. A red box highlights the 'Step 1' tool call, and a red circle with the letter 'A' is placed next to it.

IBM watsonx Orchestrate

Agent chat / Manage agents / Insurance

Model: llama-3-2-90B-v01...

Preview

Insurance 04:53 PM

Step 1

Input - 3 Lines

```
{ "customer_id": "23" }
```

Output - 3 Lines

```
{ "raw_response": "[{"id": "23", "customer_id": "2", "start_date": "2023-07-26", "end_date": "2026-07-25", "product": "boat", "premium": "1258.88"}]" }
```

Here are Sarah's policies that end within the next 30 days:

ID	Customer ID	Start Date	End Date	Product	Premium	Days to End
23	2	2026-10-15	2025-10-14	pet	380.00	20

Before step 1 the agent has correctly inferred that to find Sarah's policies it needed to use a tool, but it also reasoned that Sarah's ID could be inferred from the conversation history. In steps 2, 3 and 4 the agent is calling a tool to calculate the number of days before each one of Sarah's policies expire.

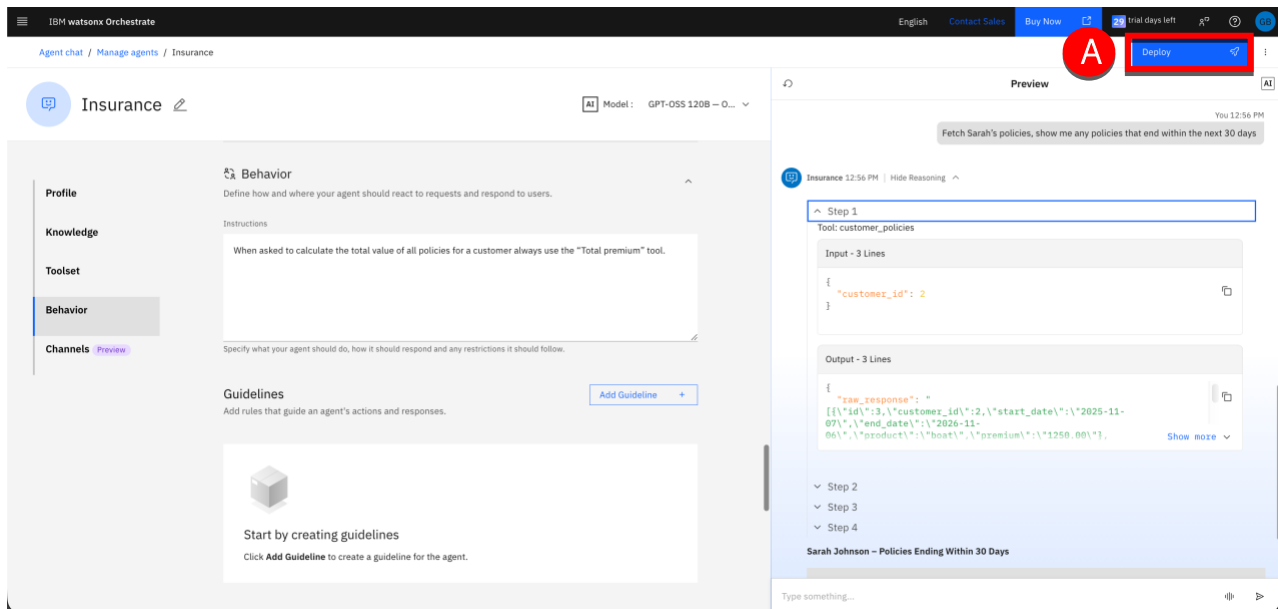
There is no tool support to find customers by name, so without this context the agent may not work. The agent may decide to do a full scan for all customers and find the first called Sarah, but this is inefficient and likely to result in an error. (The test data only contains one Sarah, in a real system there may be 1000's).

Be conscious of where your conversations rely upon context, without it, the system may respond inconsistently.

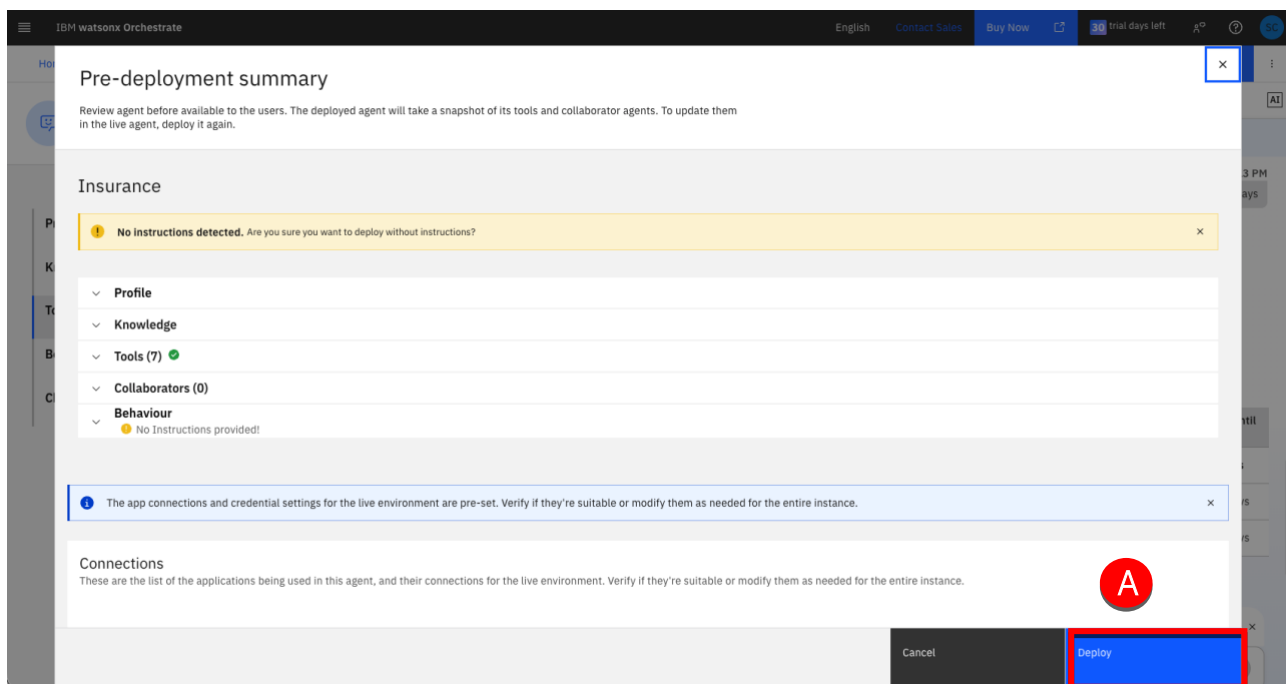
Agent deployment

In this section we will deploy the agent, making it available in the main chat service.

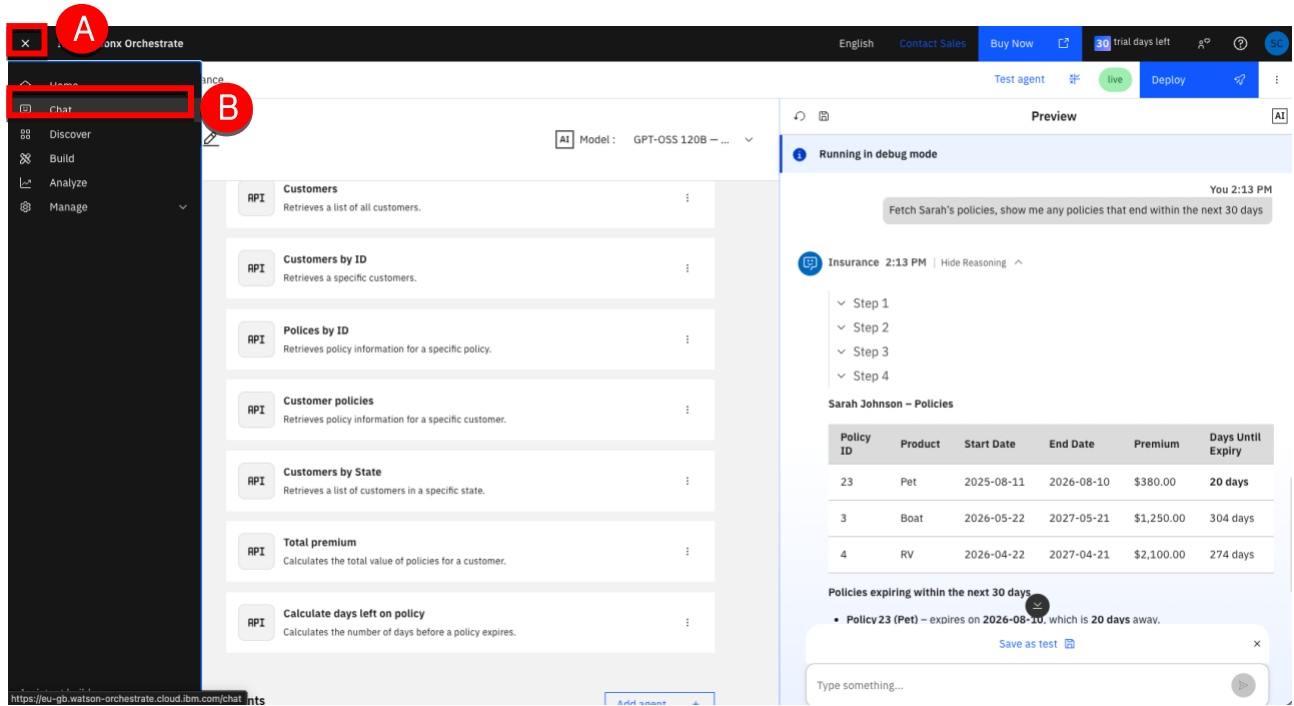
1. Click **Deploy** (A).



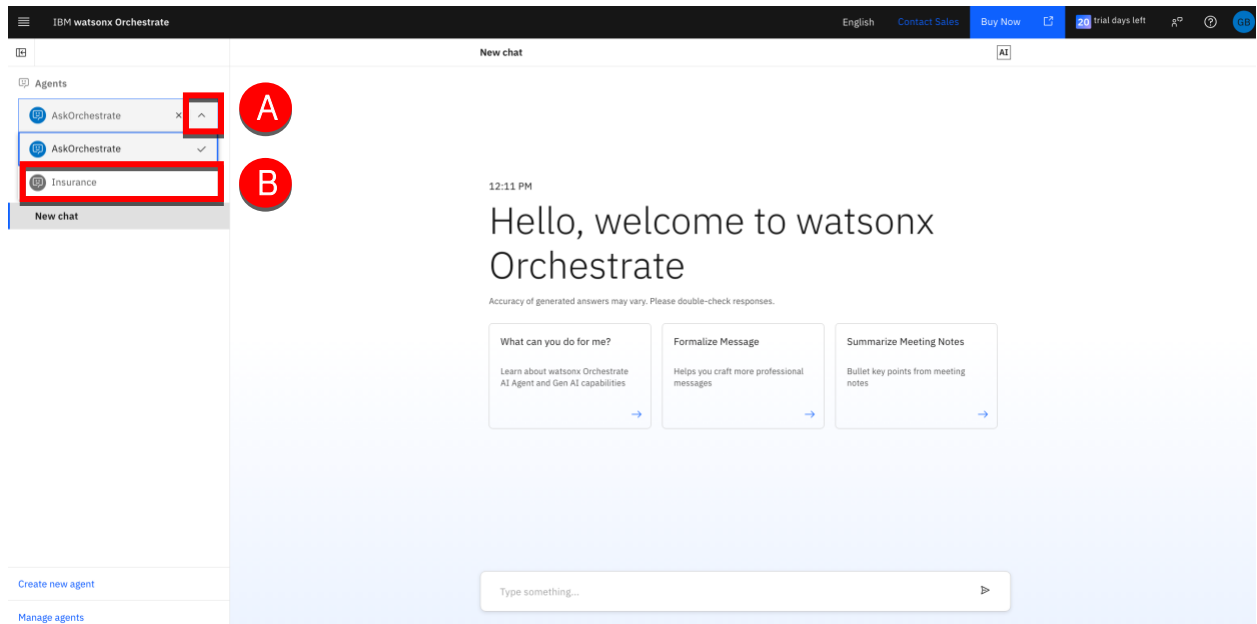
2. Click **Deploy** (A).



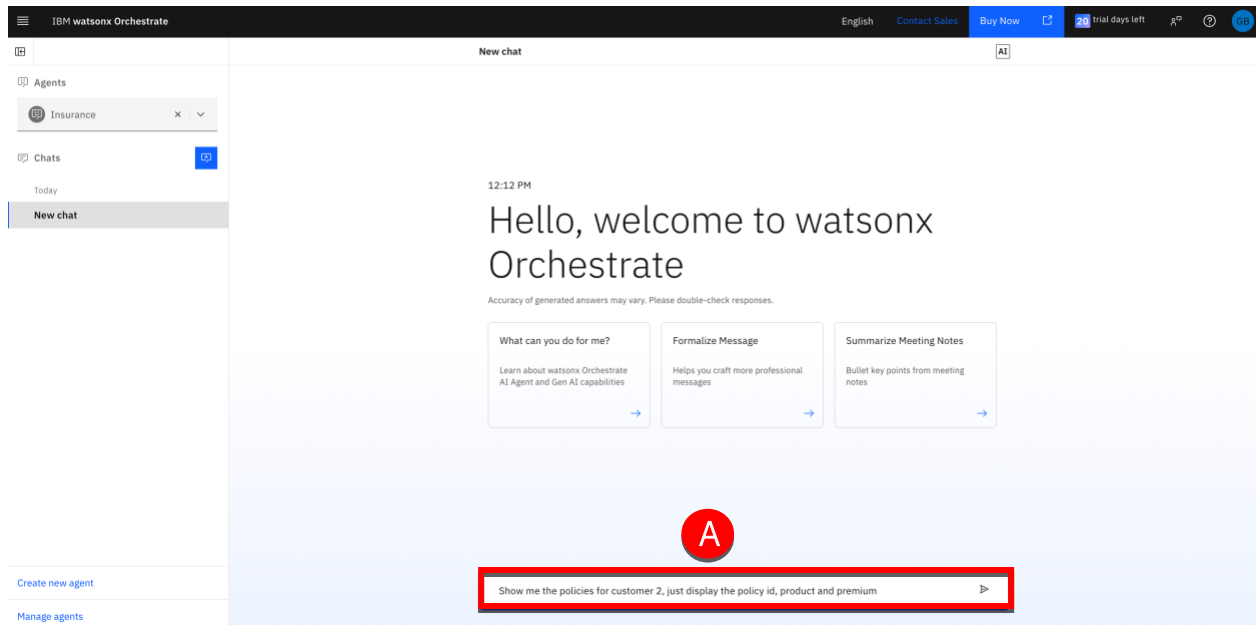
- Wait for the Success notification.
- Open the main menu (A) and click **Chat** (B).



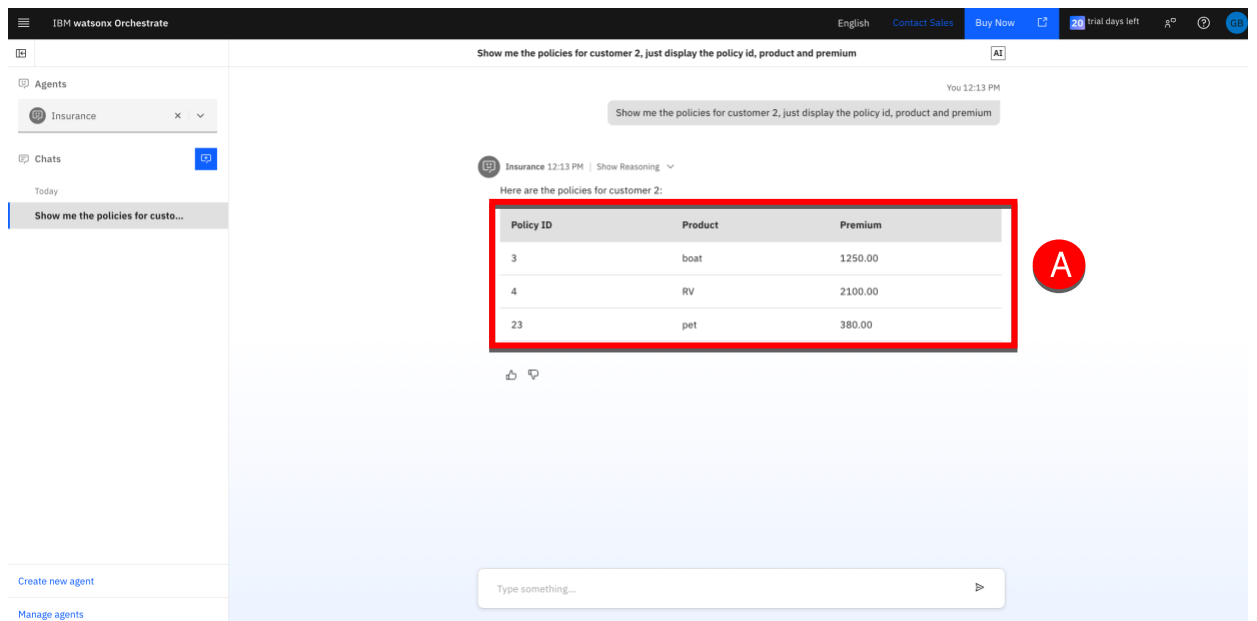
- Click the arrow adjacent to the current agent (A) to open the list of agents, then select **Insurance** (B).



6. In the chat window (A) enter “Show me the policies for customer 2, just display the policy id, product and premium” into the chat window, then press Enter.



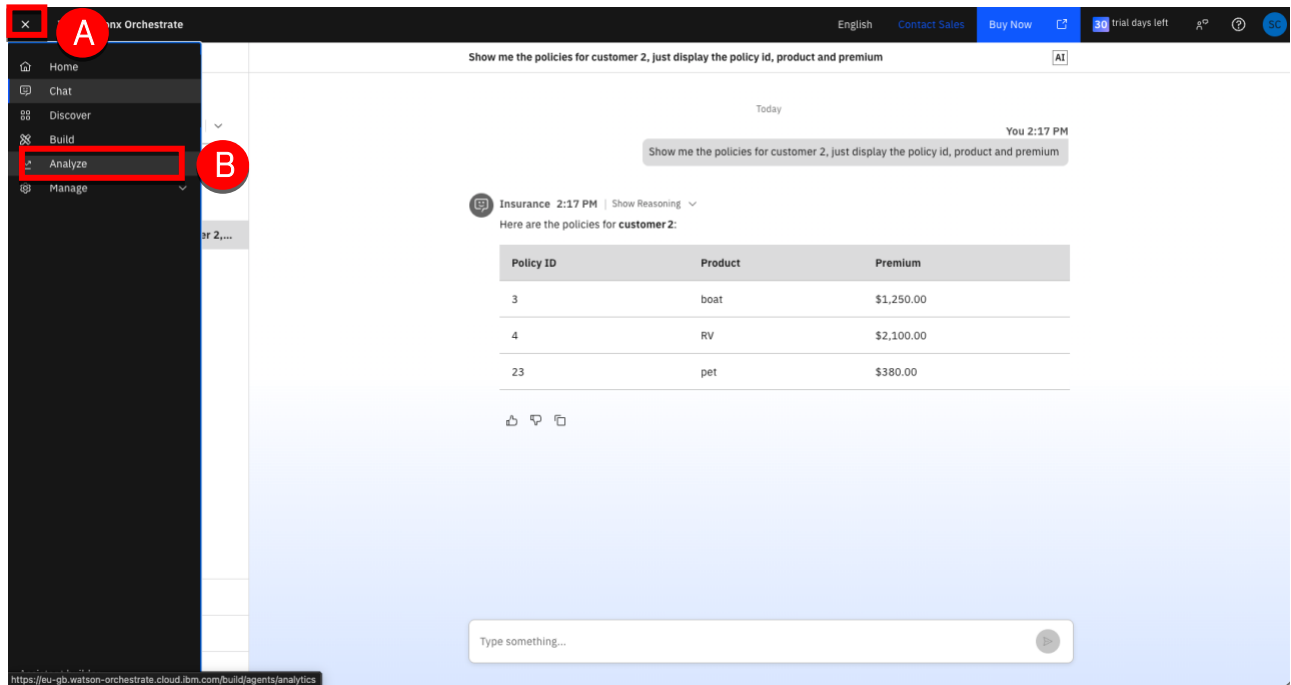
The agent responds with the policy information for customer 2 (A).



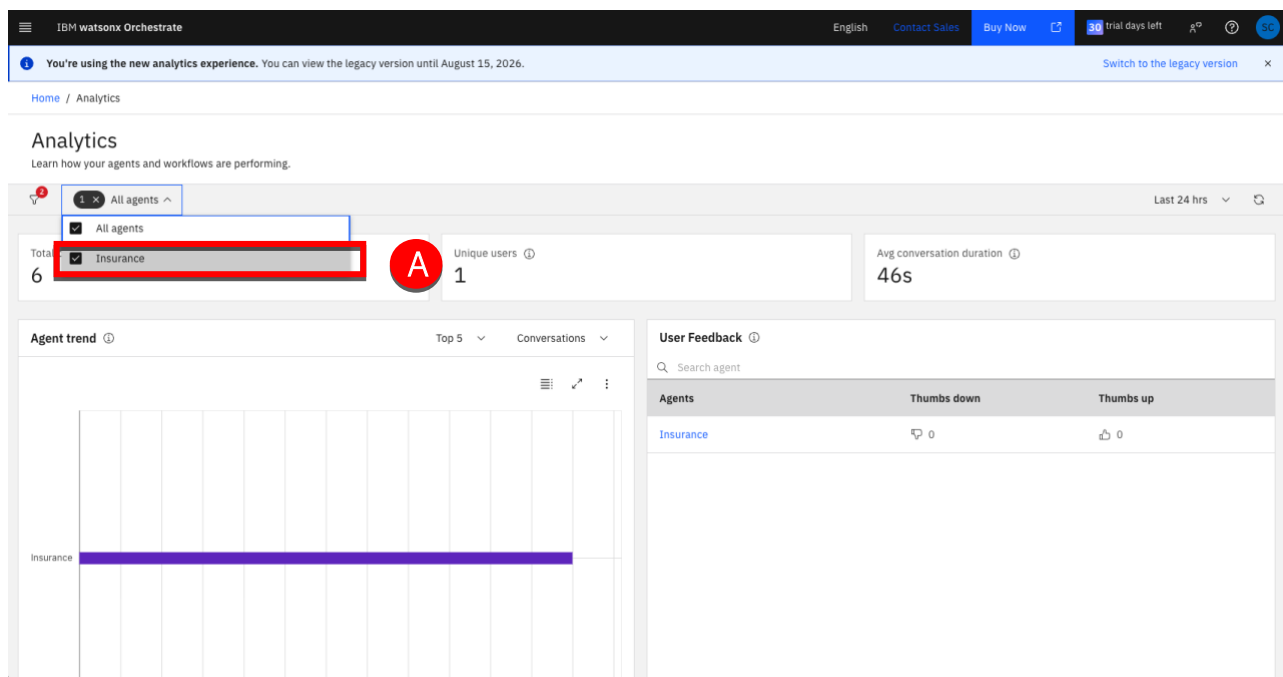
Agent analytics

In this section we will use the analytical features of watsonx Orchestrate to review agent trace information.

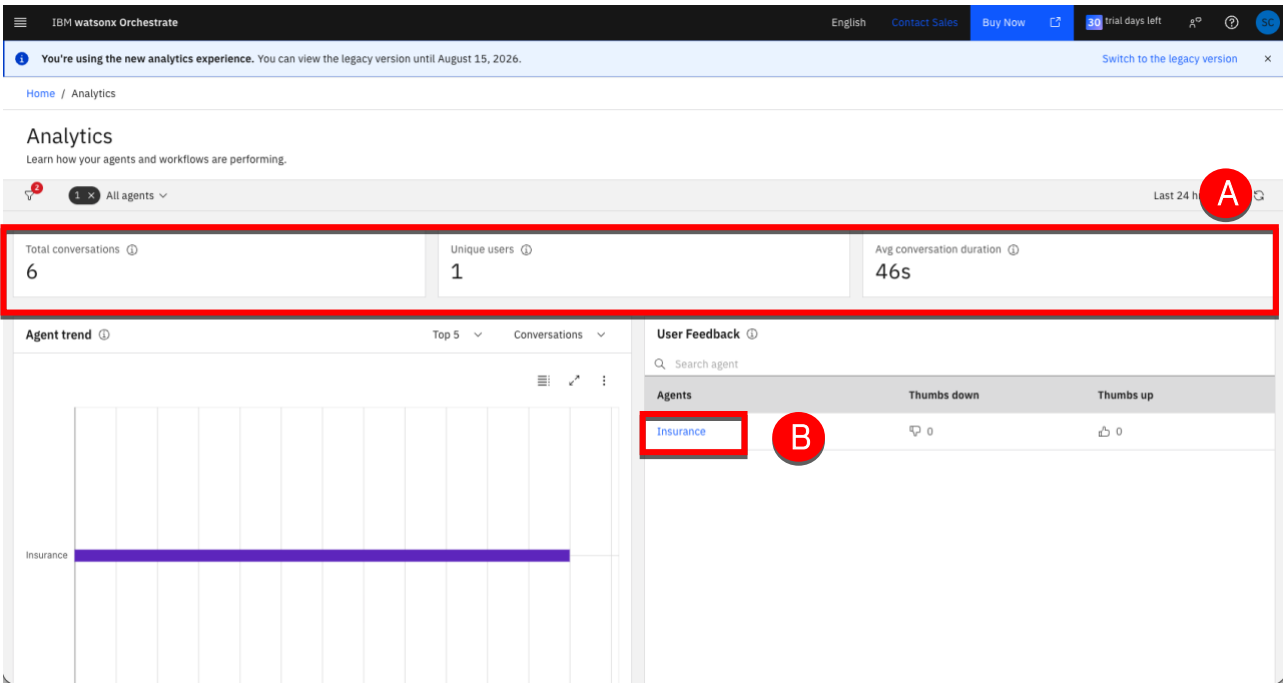
1. Open the main menu (A), then click **Analyze** (B).



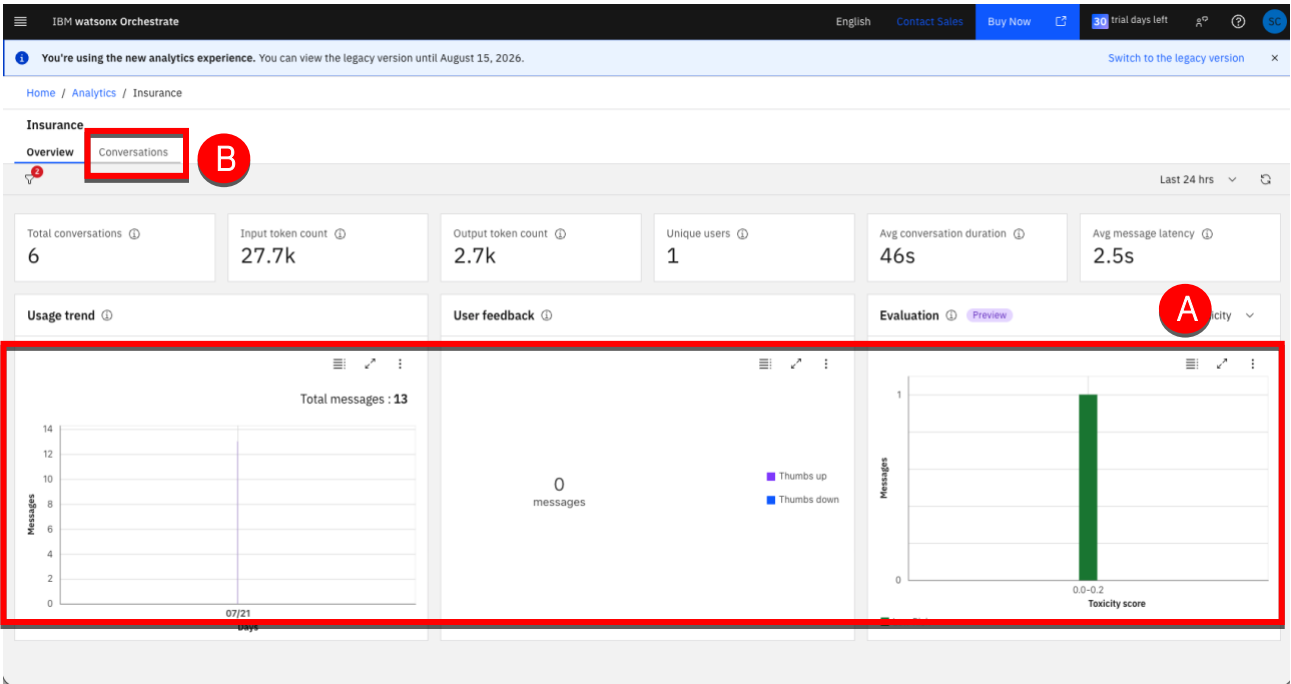
2. Select **Insurance** in the agent selector menu (A).



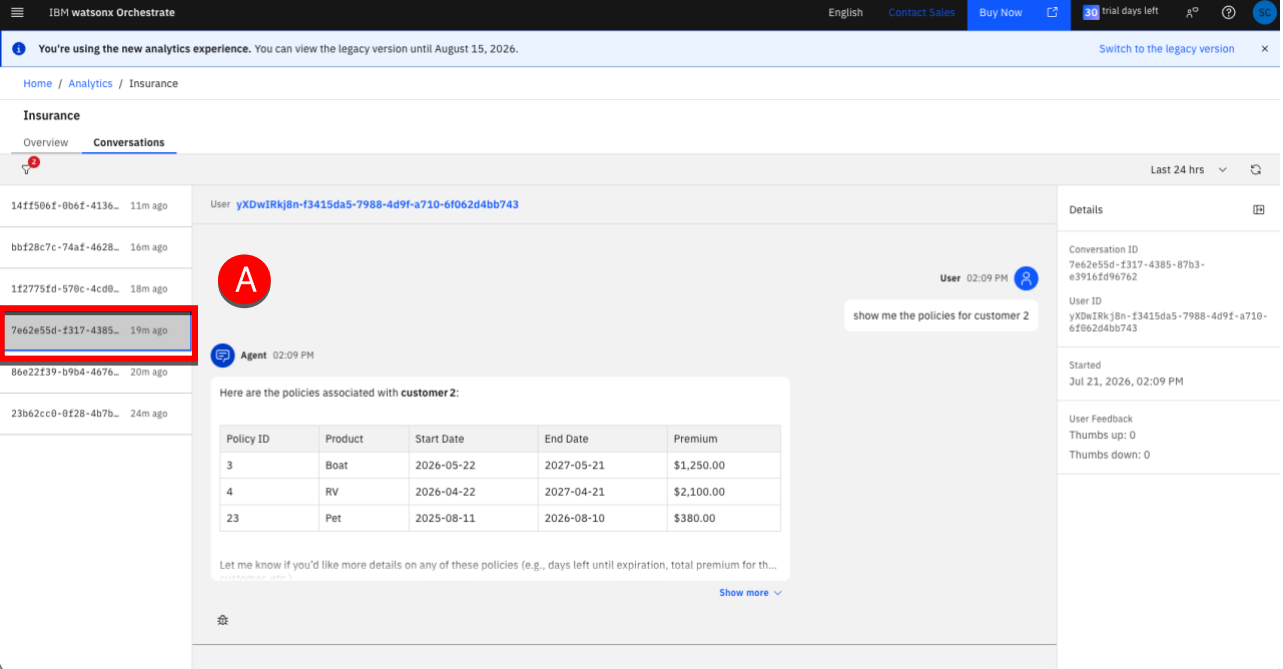
3. Note the agent statistics (A), then click on the Insurance row in the Agent feedback table.



4. Note the various statistic collected for the Insurance agent (A), click on the Conversations tab (B).



5. Select one conversation (A) and observe how the conversation history has been logged.



The screenshot displays the IBM watsonx Orchestrate interface. At the top, there's a navigation bar with 'IBM watsonx Orchestrate', language options (English, Contact Sales, Buy Now), and a trial status (38 trial days left). Below this is a banner for the new analytics experience. The main content area is titled 'Insurance' and has tabs for 'Overview' and 'Conversations'. The 'Conversations' tab is active, showing a list of conversations on the left. One conversation, with ID '7e62e55d-f317-4385...' and '19m ago', is highlighted with a red box and labeled 'A'. The right pane shows the details of this conversation. It includes a user profile (User: yXDwIRkj8n-f3415da5-7988-4d9f-a710-6f062d4bb743) and a message from the user: 'show me the policies for customer 2'. Below this, an agent response is shown: 'Here are the policies associated with customer 2:' followed by a table of policies. The table has columns: Policy ID, Product, Start Date, End Date, and Premium. The policies listed are: 3 (Boat, 2026-05-22 to 2027-05-21, \$1,250.00), 4 (RV, 2026-04-22 to 2027-04-21, \$2,100.00), and 23 (Pet, 2025-08-11 to 2026-08-10, \$380.00). A 'Show more' link is at the bottom of the table. On the far right, a 'Details' panel shows conversation metadata: Conversation ID (7e62e55d-f317-4385-87b3-e3916fd96762), User ID (yXDwIRkj8n-f3415da5-7988-4d9f-a710-6f062d4bb743), Started (Jul 21, 2026, 02:09 PM), and User Feedback (Thumbs up: 0, Thumbs down: 0).

Policy ID	Product	Start Date	End Date	Premium
3	Boat	2026-05-22	2027-05-21	\$1,250.00
4	RV	2026-04-22	2027-04-21	\$2,100.00
23	Pet	2025-08-11	2026-08-10	\$380.00

Summary

In this lab we used Agent Builder to create a basic agent able to interact with an external API. To do this, we:

- Created the agent and defined its purpose
- Imported an OpenAPI specification to create tools
- Used filtering, aggregation, and context to retrieve information
- Deployed the agent
- Reviewed the trace information recorded from our agent interactions

In subsequent labs we will build upon this knowledge and examine additional capabilities of watsonx Orchestrate.

